Plan de contrôle d'IA NetApp

NetApp Solutions

NetApp
March 19, 2024

This PDF was generated from https://docs.netapp.com/fr-fr/netapp-solutions/ai/aicp_introduction.html on March 19, 2024. Always check docs.netapp.com for the latest.

Sommaire

Plan de contrôle d'IA NetApp . . . 1
Tr-4798 : plan de contrôle pour l'IA de NetApp . . . 1
Concepts et composants . . . 2
Configuration matérielle et logicielle requise . . . 11
Déploiement Kubernetes . . . 12
Déploiement et configuration de NetApp Trident . . . 13
Déploiement Kubeflow . . . 21
Exemple d'opérations et de tâches Kubeflow . . . 30
Déploiement du flux d'air Apache . . . 38
Exemple de flux de production de flux d'air Apache . . . 41
Exemple d'opérations Trident . . . 42
Exemple d'tâches hautes performances pour les déploiements d'IA ONTAP . . . 45
Test des performances . . . 56
Conclusion . . . 56

Plan de contrôle d'IA NetApp

Tr-4798 : plan de contrôle pour l'IA de NetApp

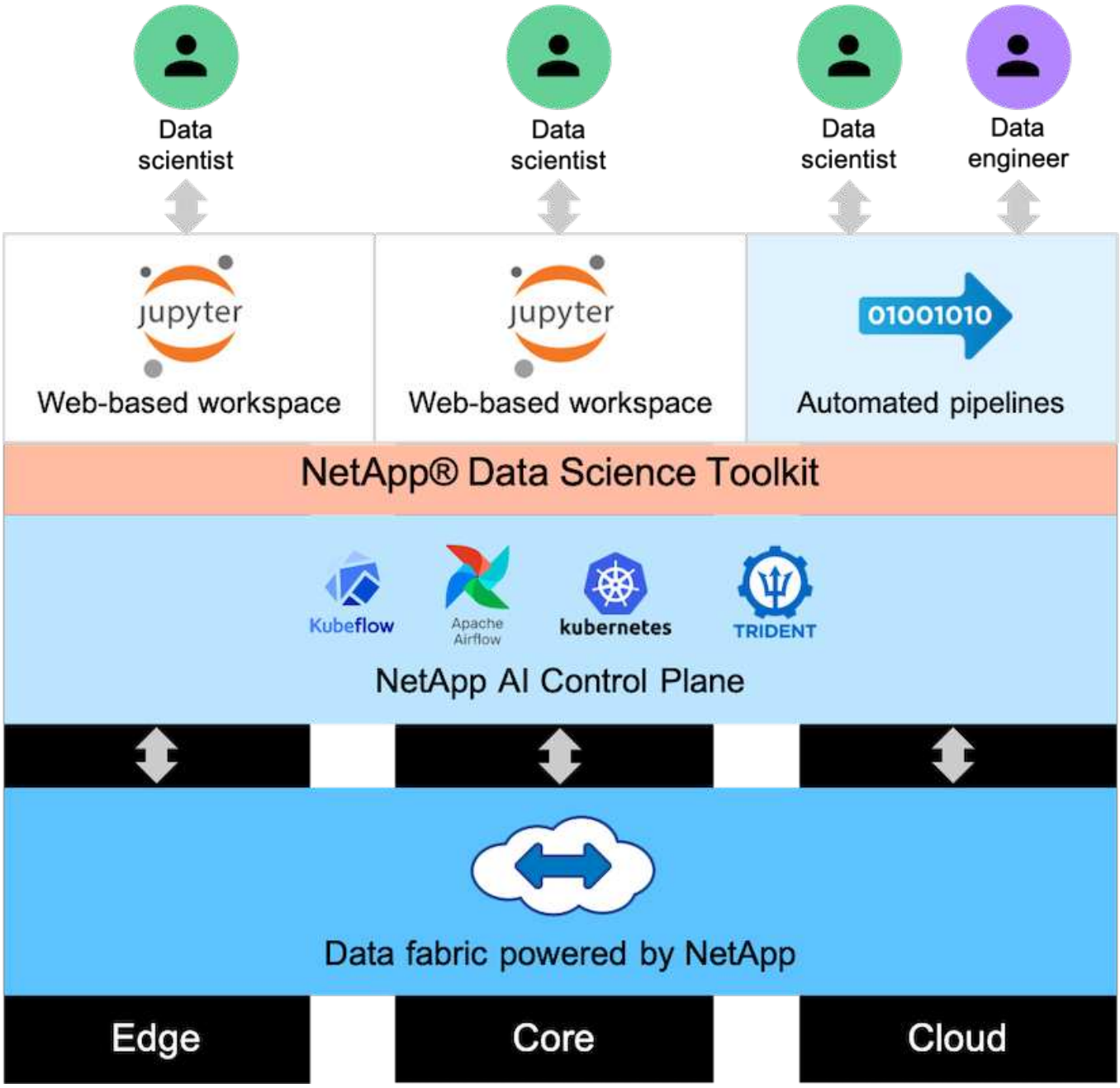
Mike Oglesby, NetApp

Toutes les entreprises, quelle que soit leur taille et leurs secteurs, se tournent vers l'intelligence artificielle (IA), le machine learning (ML) et le deep learning (DL) pour résoudre des problèmes concrets, proposer des produits et des services innovants et se démarquer sur un marché de plus en plus concurrentiel. Alors que les entreprises ont de plus en plus recours à l'IA, AU ML et au DL, elles sont confrontées à de nombreux défis, notamment l'évolutivité des workloads et la disponibilité des données. Ce document explique comment relever ces défis en utilisant NetApp ai Control plane, une solution qui associe les fonctionnalités de gestion des données NetApp aux outils et frameworks open source les plus répandus.

Ce rapport vous explique comment cloner rapidement un espace de noms de données. Il vous montre également comment répliquer de manière fluide les données entre plusieurs sites et régions pour créer un pipeline de données d'IA/AM/AP cohérent et unifié. Vous y trouverez également des informations sur la définition et l'implémentation des workflows d'entraînement d'IA, DE ML et de DL pour créer quasi instantanément des données et des modèles de base à des fins de traçabilité et de gestion des versions. Avec cette solution, vous pouvez suivre l'entraînement des modèles jusqu'au dataset exact où on a utilisé pour entraîner et/ou valider le modèle. Enfin, ce document vous montre comment provisionner rapidement des espaces de travail Jupyter Notebook avec accès à des jeux de données volumineux.

Remarque : pour un entraînement distribué de style HPC à grande échelle impliquant un grand nombre de serveurs GPU qui nécessitent un accès partagé au même dataset, ou si vous avez besoin/préférez un système de fichiers parallèle, consultez le document "TR-4890". Ce rapport technique décrit comment inclure "La solution de système de fichiers parallèle entièrement prise en charge par NetApp, BeeGFS" Dans le cadre du plan de contrôle d'IA NetApp. Cette solution est conçue pour s'adapter de quelques systèmes NVIDIA DGX A100 à un superPOD de 140 nœuds.

Le plan de contrôle de l'IA de NetApp est destiné aux data Scientists et aux ingénieurs de données. L'expertise minimale de NetApp ou de NetApp ONTAP® est donc requise. Avec cette solution, les fonctions de gestion des données peuvent être exécutées à l'aide d'outils et d'interfaces simples et familiers. Si votre environnement dispose déjà d'un système de stockage NetApp, vous pouvez tester le plan NetApp ai Control dès aujourd'hui. Si vous souhaitez tester la solution, mais que vous ne disposez pas encore de système de stockage NetApp, rendez-vous sur "cloud.netapp.com", Vous pouvez être opérationnel avec une solution de stockage NetApp dans le cloud en quelques minutes. La figure suivante fournit une visualisation de la solution.



Concepts et composants

L'intelligence artificielle

L'IA est une discipline scientifique informatique dans laquelle les ordinateurs sont entraînés pour imiter les fonctions cognitives de l'esprit humain. Les développeurs d'IA entraînent des ordinateurs pour apprendre et résoudre les problèmes de manière similaire, voire supérieure, à celle des humains. L'apprentissage profond et l'apprentissage machine sont des sous-domaines de l'IA. Les entreprises adoptent de plus en plus l'IA, LE ML et le DL pour répondre à leurs besoins stratégiques. Voici quelques exemples :

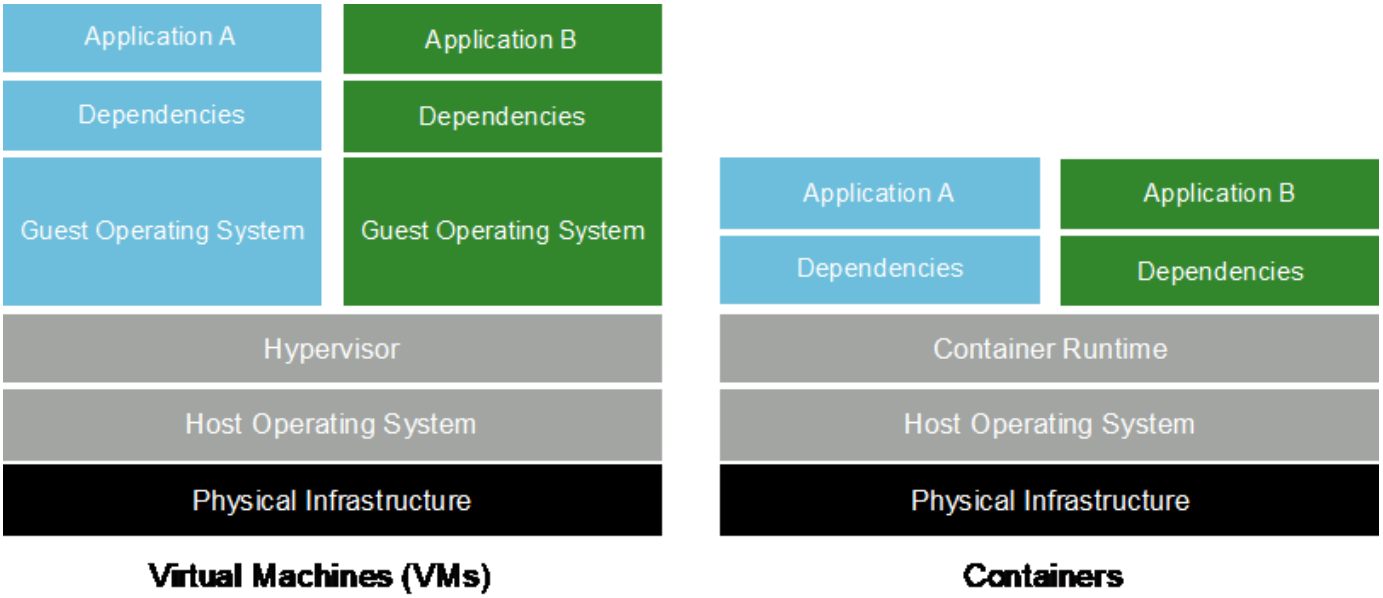
- Analyser d'importants volumes de données pour obtenir des informations stratégiques inexistantes
- Interagir directement avec les clients à l'aide du traitement du langage naturel
- Automatisation de divers processus et fonctions métier

Les workloads d'entraînement et d'inférence d'IA modernes requièrent des fonctionnalités de calcul parallèle. Par conséquent, les GPU sont de plus en plus utilisés pour exécuter les opérations d'IA, car les capacités de traitement parallèle des GPU sont largement supérieures à celles des processeurs génériques.

Conteneurs

Les conteneurs sont des instances isolées de l'espace utilisateur qui s'exécutent sur un noyau de système d'exploitation hôte partagé. Les conteneurs se généraaffichent de plus en plus rapidement. Les conteneurs offrent bon nombre des avantages de la boxe applicative offerts par les machines virtuelles. Cependant, les couches de l'hyperviseur et du système d'exploitation invité sur lesquelles reposent les machines virtuelles ont été éliminées, les conteneurs sont beaucoup plus légers. La figure suivante montre une visualisation des machines virtuelles par rapport aux conteneurs.

Les conteneurs permettent également de packaging efficace des dépendances entre applications, des durées d'exécution, etc., directement avec une application. Le format de conditionnement de conteneurs le plus utilisé est le container Docker. Une application conteneurisée dans le format de conteneur Docker peut être exécutée sur n'importe quel ordinateur capable d'exécuter des conteneurs Docker. Cela est vrai même si les dépendances de l'application ne sont pas présentes sur la machine car toutes les dépendances sont conditionnées dans le conteneur lui-même. Pour plus d'informations, consultez la "Site Web de Docker".



Kubernetes

Kubernetes est une plateforme open source d'orchestration de conteneurs distribuée, conçue à l'origine par Google, et désormais gérée par Cloud Native Computing Foundation (CNCF). Kubernetes permet d'automatiser le déploiement, la gestion et l'évolutivité des fonctions pour les applications conteneurisées. Depuis quelques années, Kubernetes devient la plateforme dominante d'orchestration de conteneurs. Bien que les autres formats de conditionnement et les temps d'exécution des conteneurs soient pris en charge, Kubernetes est le plus souvent utilisé comme système d'orchestration pour les conteneurs Docker. Pour plus d'informations, consultez la "Site Web de Kubernetes".

NetApp Trident

Trident est un orchestrateur de stockage open source développé et géré par NetApp qui simplifie considérablement la création, la gestion et la consommation du stockage persistant pour les workloads Kubernetes. Trident, une application Kubernetes native, s'exécute directement dans un cluster Kubernetes.

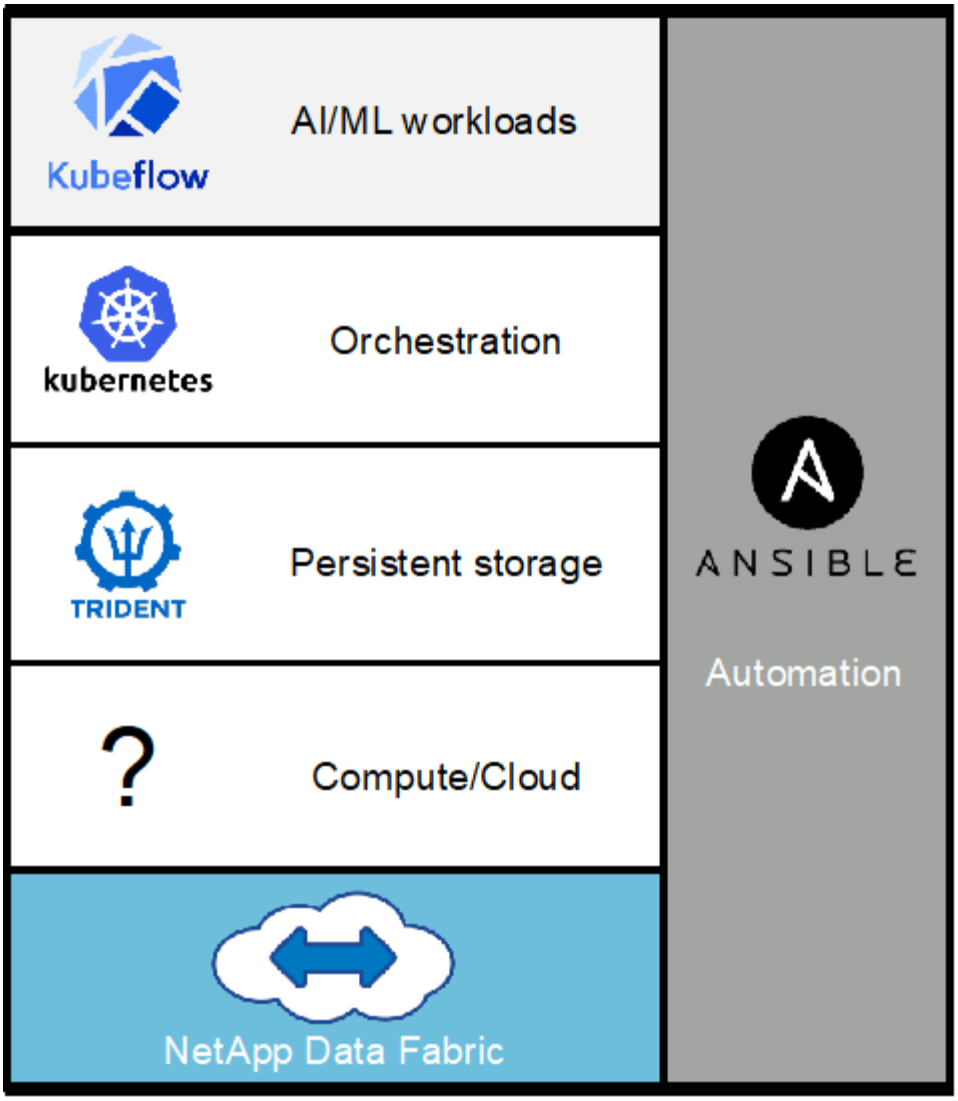
Avec Trident, les utilisateurs de Kubernetes (développeurs, data Scientists, administrateurs Kubernetes, etc.) peuvent créer, gérer et interagir avec les volumes de stockage persistant dans le format Kubernetes standard qu'ils connaissent déjà. Ils peuvent également bénéficier des fonctionnalités avancées de gestion des données de NetApp et d'un environnement Data Fabric optimisé par la technologie NetApp. Trident élimine les complexités du stockage persistant et facilite la consommation. Pour plus d'informations, consultez la "Site Web Trident".

NVIDIA DeepOps

DeepOps est un projet open source de NVIDIA qui, grâce à Ansible, automatise le déploiement de clusters de serveurs GPU conformément aux bonnes pratiques. DeepOps est modulaire et peut être utilisé pour diverses tâches de déploiement. Pour ce document et l'exercice de validation décrit, DeepOps est utilisé pour déployer un cluster Kubernetes composé de nœuds workers de serveurs GPU. Pour plus d'informations, consultez la "Site Web de DeepOps".

Kubeflow

Kubeflow est un kit d'IA et DE ML open source pour Kubernetes qui a été développé à l'origine par Google. Le projet Kubeflow permet de déployer des workflows d'IA et DE ML sur Kubernetes de façon simple, portable et évolutive. Kubeflow s'affranchit des complexités de Kubernetes, ce qui permet aux data Scientists de se concentrer sur les meilleures connaissances—data science. Voir la figure suivante pour une visualisation. Kubeflow s'est beaucoup plus à l'mesure que les services IT d'entreprise sont de plus en plus standardisés sur Kubernetes. Pour plus d'informations, consultez la "Site web Kubeflow".



Kubeflow pipelines

Kubeflow pipelines est un composant clé de Kubeflow. Kubeflow pipelines est une plateforme et une norme pour définir et déployer des workflows d'IA et DE ML portables et évolutifs. Pour plus d'informations, reportez-vous à la section "Documentation officielle Kubeflow".

Jupyter Notebook Server

Un Jupyter Notebook Server est une application web open source qui permet aux data Scientists de créer des documents de type wiki appelés ordinateurs portables Jupyter contenant du code en direct ainsi que des tests descriptifs. Les ordinateurs portables Jupyter sont largement utilisés dans la communauté de l'IA et DU ML afin de documenter, de stocker et de partager des projets d'IA et DE ML. Kubeflow simplifie le provisionnement et le déploiement de Jupyter Notebooks Servers sur Kubernetes. Pour plus d'informations sur les ordinateurs portables Jupyter, visitez le "Site Web de Jupyter". Pour plus d'informations sur les ordinateurs portables Jupyter dans le contexte de Kubeflow, consultez le "Documentation officielle Kubeflow".

Débit d'air Apache

Apache Airflow est une plateforme de gestion des flux de production open source qui permet de créer des programmes, de planifier et de surveiller des flux de travail d'entreprise complexes. Il est souvent utilisé pour automatiser les workflows ETL et de pipeline de traitement de données, mais ne se limite pas à ces types de

flux de travail. Le projet de flux d'air a été lancé par Airbnb mais est depuis devenu très populaire dans l'industrie et est maintenant sous les auspices de la Apache Software Foundation. Le flux d'air est écrit en Python, les flux de production du flux d'air sont créés via des scripts Python, et le flux d'air est conçu selon le principe de « configuration as code ». De nombreux utilisateurs de flux d'air utilisent désormais Kubernetes.

Graphes acycliques dirigés (DAG)

Dans le flux d'air, les flux de travail sont appelés graphiques acycliques dirigés (DAG). Les GDAs sont constitués de tâches exécutées dans l'ordre, en parallèle ou une combinaison des deux, en fonction de la définition DAG. Le planificateur de flux d'air exécute des tâches individuelles sur un ensemble de travailleurs, en respectant les dépendances au niveau des tâches spécifiées dans la définition DAG. Les DAG sont définis et créés par le biais de scripts Python.

NetApp ONTAP 9

NetApp ONTAP 9 est la dernière génération de logiciel de gestion du stockage de NetApp qui permet à des entreprises comme la vôtre de moderniser l'infrastructure et de passer à un data Center prêt pour le cloud. Avec des capacités de gestion des données à la pointe du secteur, ONTAP vous permet de gérer et de protéger vos données avec un seul ensemble d'outils, quel que soit leur emplacement. Vous pouvez aussi déplacer vos données librement partout où vous en avez besoin : la périphérie, le cœur ou le cloud. ONTAP 9 comprend de nombreuses fonctionnalités qui simplifient la gestion des données, accélèrent et protègent les données stratégiques et pérennisent l'infrastructure sur toutes les architectures de cloud hybride.

Simplifiez la gestion des données

La gestion des données est cruciale pour les opérations IT, car elle vous permet d'utiliser les ressources appropriées pour vos applications et vos datasets. ONTAP inclut les fonctionnalités suivantes pour rationaliser et simplifier vos opérations et réduire le coût total d'exploitation :

- **La compaction des données à la volée et la déduplication étendue** la compaction des données réduit le gaspillage d'espace dans les blocs de stockage, et la déduplication augmente considérablement la capacité effective.
- **Qualité de service (QoS) minimale, maximale et adaptative.** les contrôles de qualité de service granulaires permettent de maintenir les niveaux de performance des applications critiques dans des environnements hautement partagés.
- **ONTAP FabricPool** cette fonctionnalité permet la hiérarchisation automatique des données inactives vers des options de stockage en cloud public et privé, notamment Amazon Web Services (AWS), Azure et le stockage objet NetApp StorageGRID.

Accélération et protection des données

ONTAP offre des niveaux supérieurs de performances et de protection des données et étend ces fonctionnalités grâce à plusieurs fonctionnalités :

- * Hautes performances et faible latence.* ONTAP offre le débit le plus élevé possible à la latence la plus faible possible.
- **Technologie NetApp ONTAP FlexGroup** Un volume FlexGroup est un conteneur de données haute performance pouvant évoluer de manière linéaire jusqu'à 20 po et 400 milliards de fichiers, fournissant un espace de noms unique qui simplifie la gestion des données.
- **Protection des données.** ONTAP fournit des fonctionnalités de protection des données intégrées avec une gestion commune sur toutes les plates-formes.
- **NetApp Volume Encryption**. ONTAP offre un chiffrement natif au niveau du volume avec prise en charge

de la gestion des clés à la fois intégrée et externe.

Une infrastructure pérenne

ONTAP 9 aide à répondre aux besoins métier en constante évolution :

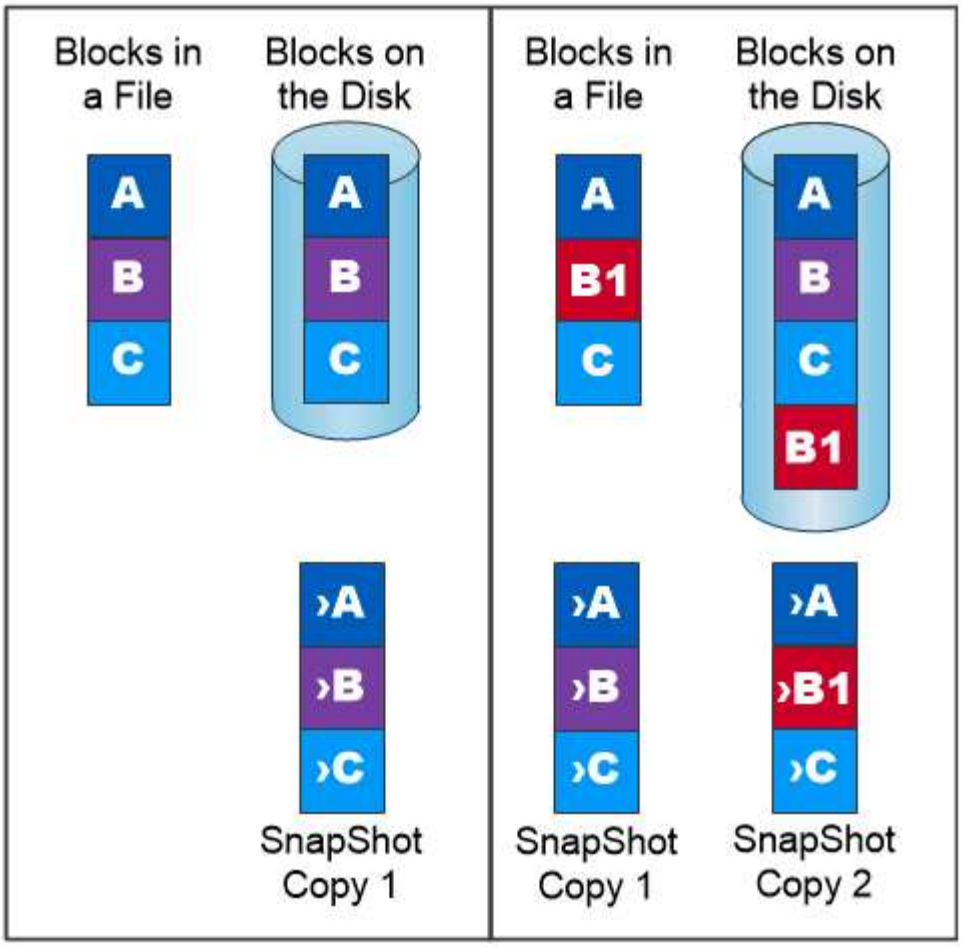
- **Évolutivité transparente et continuité de l'activité.** ONTAP prend en charge l'ajout non disruptif de capacité aux contrôleurs et l'évolution scale-out des clusters. Vous pouvez effectuer la mise à niveau vers les technologies les plus récentes, telles que NVMe et FC 32 Gb, sans migration des données ni panne coûteuse.
- **Connexion au cloud.** ONTAP est l'un des logiciels de gestion de stockage les plus connectés au cloud, avec des options de stockage SDS (ONTAP Select) et des instances natives du cloud (NetApp Cloud Volumes Service) dans tous les clouds publics.
- **Intégration avec les applications émergentes** en utilisant la même infrastructure qui prend en charge les applications d'entreprise existantes, ONTAP propose des services de données haute performance pour les plateformes et applications nouvelle génération, comme OpenStack, Hadoop et MongoDB.

Copies NetApp Snapshot

Une copie NetApp Snapshot est une image ponctuelle en lecture seule d'un volume. La consommation d'espace de stockage de l'image est minime et l'impact sur les performances est négligeable, car elle enregistre uniquement les modifications apportées aux fichiers depuis la dernière copie Snapshot, comme illustré dans la figure ci-dessous.

Les copies Snapshot doivent optimiser leur efficacité par rapport à la technologie de virtualisation de base du stockage ONTAP, WAFL (Write Anywhere File Layout). Tout comme une base de données, WAFL utilise des métadonnées pour désigner des blocs de données réels sur le disque. Contrairement à une base de données, WAFL ne remplace pas les blocs existants. Il écrit les données mises à jour sur un nouveau bloc et modifie les métadonnées. C'est parce que ONTAP référence les métadonnées lorsqu'il crée une copie Snapshot, plutôt que de copier des blocs de données, ces copies sont si efficaces. Vous éliminez ainsi les temps de recherche engendrés par d'autres systèmes pour localiser les blocs à copier, et par ailleurs le coût d'une copie.

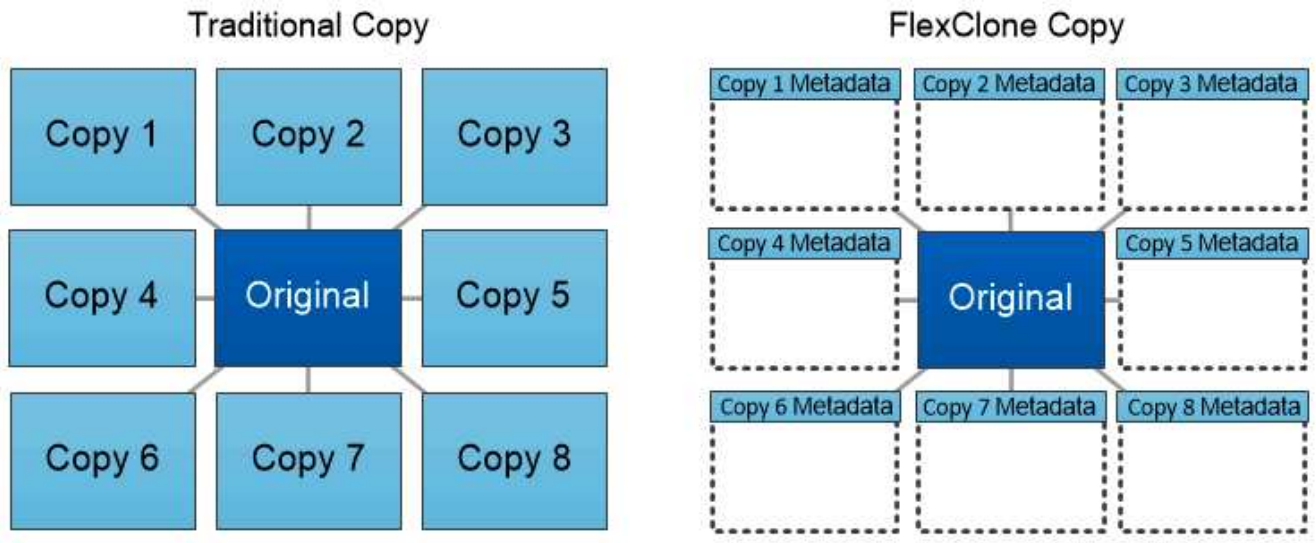
Vous pouvez utiliser une copie Snapshot pour restaurer des fichiers ou des LUN individuels, ou pour restaurer l'ensemble du contenu d'un volume. ONTAP compare les informations du pointeur de la copie Snapshot aux données d'un disque pour reconstruire l'objet manquant ou endommagé, sans temps d'indisponibilité ni coûts de performance significatifs.



A Snapshot copy records only changes to the active file system since the last Snapshot copy.

Technologie NetApp FlexClone

La technologie NetApp FlexClone référence les métadonnées Snapshot pour créer des copies inscriptibles instantanées d'un volume. Les copies partagent les blocs de données avec leurs parents. Aucun stockage n'est utilisé, sauf pour les métadonnées, jusqu'à ce que les modifications soient écrites sur la copie, comme illustré dans la figure ci-dessous. Là où les copies classiques peuvent prendre des minutes, voire des heures, pour créer des copies, FlexClone vous permet de copier même les jeux de données les plus volumineux quasi instantanément. C'est pourquoi il est idéal si vous avez besoin de plusieurs copies de jeux de données identiques (un espace de travail de développement, par exemple) ou de copies temporaires d'un jeu de données (afin de tester une application par rapport à un jeu de données de production).

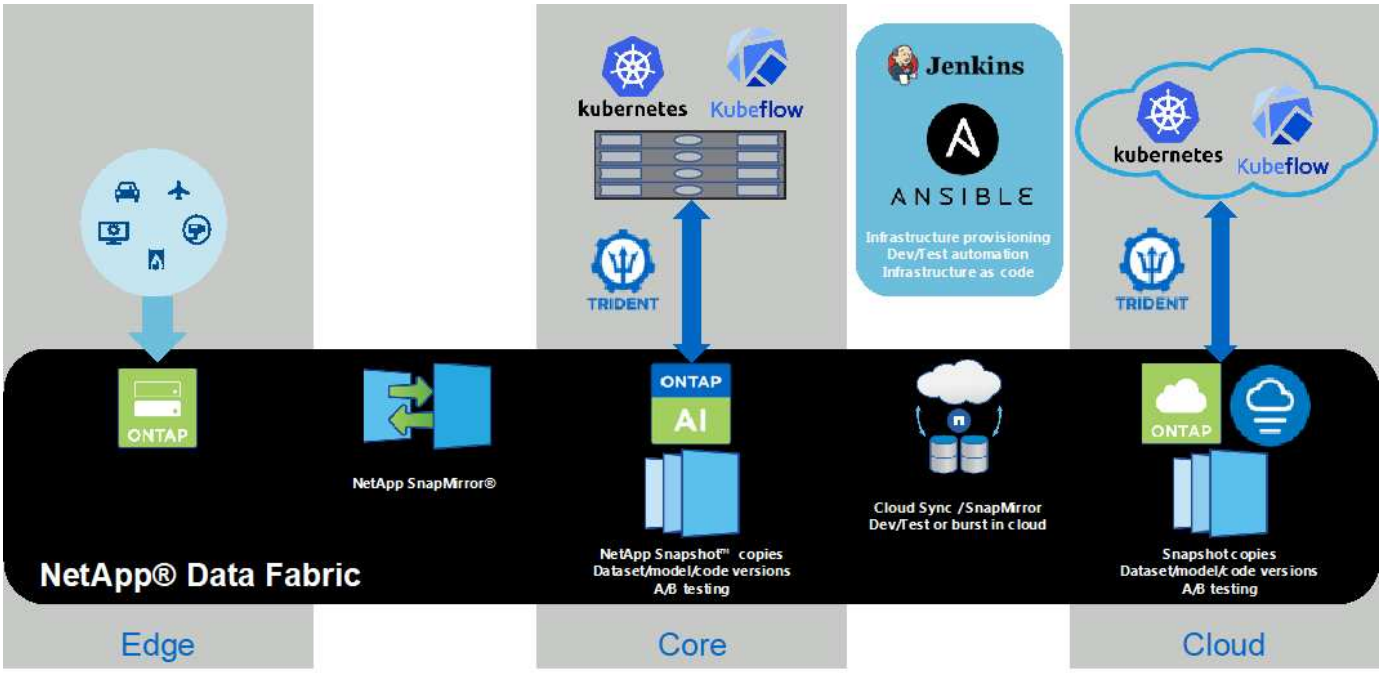


FlexClone copies share data blocks with their parents, consuming no storage except what is required for metadata.

Technologie de réplication des données NetApp SnapMirror

Le logiciel NetApp SnapMirror est une solution de réplication unifiée économique et facile à utiliser dans l'environnement Data Fabric. Il réplique les données à haute vitesse sur un WAN ou un LAN. Elle vous assure haute disponibilité et une réplication rapide des données pour les applications de tous types, y compris les applications stratégiques dans les environnements classiques et virtuels. En répliquant vos données sur un ou plusieurs systèmes de stockage NetApp, puis en les mettant régulièrement à jour, vous disposez de données actualisées et accessibles dès que vous en avez besoin. Aucun serveur de réplication externe n'est requis. Voir la figure suivante pour un exemple d'architecture exploitant la technologie SnapMirror.

Le logiciel SnapMirror valorise l'efficacité du stockage NetApp ONTAP en n'envoyant que les blocs modifiés sur le réseau. Il utilise également la compression réseau intégrée pour accélérer le transfert de données et réduire l'utilisation de la bande passante jusqu'à 70 %. Avec la technologie SnapMirror, vous pouvez exploiter un flux de données de réplication fine pour créer un référentiel unique qui administre les copies du miroir actif et les copies instantanées antérieures, réduisant ainsi le trafic du réseau jusqu'à 50 %.



Copie et synchronisation NetApp BlueXP

BlueXP Copy and Sync est un service NetApp qui permet une synchronisation sûre et rapide des données. Qu'il s'agisse de transférer des fichiers entre des partages de fichiers NFS ou SMB sur site, NetApp StorageGRID, NetApp ONTAP S3, NetApp Cloud Volumes Service, Azure NetApp Files, AWS S3, AWS EFS, Azure Blob, Google Cloud Storage ou IBM Cloud Object Storage, BlueXP Copy and Sync déplace les fichiers où vous le souhaitez, rapidement et en toute sécurité.

Une fois vos données transférées, elles peuvent être utilisées à la source et à la cible. BlueXP Copy and Sync peut synchroniser des données à la demande lorsqu'une mise à jour est déclenchée ou lorsque les données sont synchronisées en continu sur la base d'un calendrier prédéfini. Quoi qu'il en soit, la copie et la synchronisation BlueXP ne déplacent que les données modifiées, ce qui réduit le temps et l'argent consacrés à la réplication des données.

BlueXP Copy and Sync est un outil SaaS extrêmement simple à configurer et à utiliser. Les transferts de données déclenchés par la copie et la synchronisation BlueXP sont effectués par les courtiers de données. Les courtiers de données BlueXP Copy and Sync peuvent être déployés dans AWS, Azure, Google Cloud Platform ou sur site.

NetApp XCP

NetApp XCP est un logiciel client pour les migrations de données et les informations relatives au système de fichiers de tout type à NetApp et NetApp à NetApp. XCP a été conçu pour évoluer et atteindre des performances maximales en exploitant toutes les ressources système disponibles pour gérer des datasets à grand volume et des migrations haute performance. XCP vous aide à obtenir une visibilité complète sur le système de fichiers avec la possibilité de générer des rapports.

NetApp XCP est disponible dans un pack unique qui prend en charge les protocoles NFS et SMB. XCP inclut un binaire Linux pour les jeux de données NFS et un exécutable Windows pour les jeux de données SMB.

NetApp XCP File Analytics est un logiciel basé sur l'hôte qui détecte les partages de fichiers, exécute les analyses sur le système de fichiers et fournit un tableau de bord pour l'analytique des fichiers. XCP File Analytics est compatible avec les systèmes NetApp et non NetApp et s'exécute sur des hôtes Linux ou

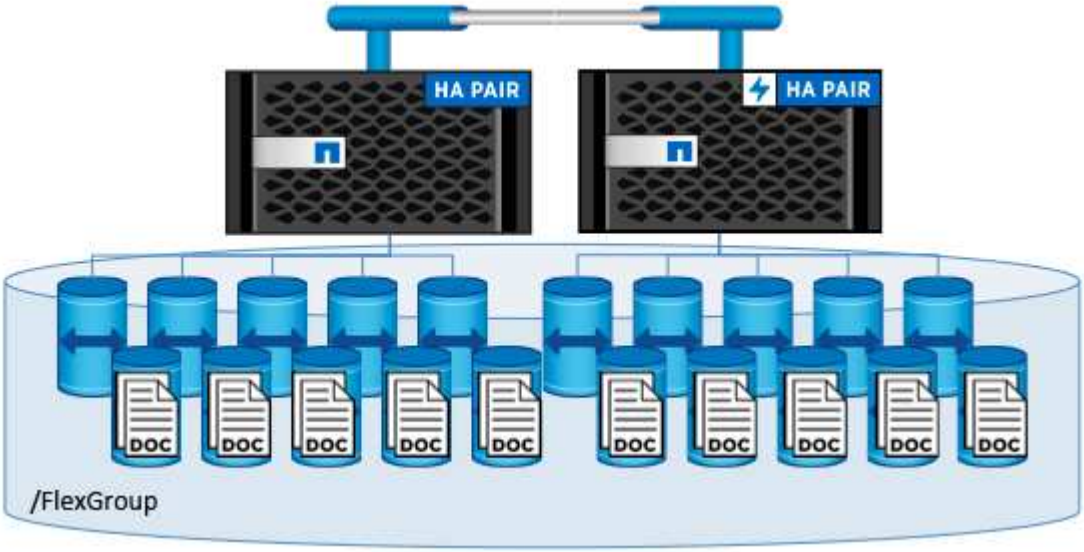
Windows pour fournir des analyses des systèmes de fichiers exportés NFS et SMB.

NetApp ONTAP FlexGroup volumes

Un dataset d'entraînement peut être un ensemble de milliards de fichiers. Les fichiers peuvent inclure du texte, de l'audio, de la vidéo et d'autres formes de données non structurées qui doivent être stockées et traitées pour être lues en parallèle. Le système de stockage doit stocker un grand nombre de petits fichiers et doit lire ces fichiers en parallèle pour les E/S séquentielles et aléatoires

Un volume FlexGroup est un namespace unique qui comprend plusieurs volumes de membres constitutifs, comme illustré dans la figure suivante. Du point de vue de l'administrateur de stockage, un volume FlexGroup est géré et agit comme un volume NetApp FlexVol. Les fichiers du volume FlexGroup sont alloués aux volumes de membres individuels,et non répartis entre les volumes ou les nœuds. Ils présentent de nombreux atouts :

- Les volumes FlexGroup fournissent une capacité de plusieurs pétaoctets et une faible latence prévisible pour les charges de travail comportant un grand nombre de métadonnées.
- Ils prennent en charge jusqu'à 400 milliards de fichiers dans le même namespace.
- Ils prennent en charge les opérations parallélisées dans les charges de travail NAS sur les processeurs, les nœuds, les agrégats et les volumes FlexVol constitutifs.



Configuration matérielle et logicielle requise

La solution NetApp ai Control plane ne dépend pas de ce matériel spécifique. La solution est compatible avec toute appliance de stockage physique, instance Software-defined ou service cloud NetApp, prise en charge par Trident. À titre d'exemple, on peut citer un système de stockage NetApp AFF, Azure NetApp Files, NetApp Cloud Volumes Service, une instance de stockage Software-defined NetApp ONTAP Select ou une instance NetApp Cloud Volumes ONTAP. De plus, la solution peut être implémentée sur n'importe quel cluster Kubernetes, tant que la version Kubernetes utilisée est prise en charge par Kubeflow et NetApp Trident. Pour obtenir la liste des versions Kubernetes prises en charge par Kubeflow, voir la "Documentation officielle Kubeflow". Pour obtenir la liste des

versions de Kubernetes prises en charge par Trident, consultez le "Documentation Trident". Pour plus de détails sur l'environnement utilisé pour valider la solution, reportez-vous aux tableaux suivants.

Composant d'infrastructure	Quantité	Détails	Système d'exploitation
Hôte de démarrage du déploiement	1	VM	Ubuntu 20.04.2 LTS
Nœuds maîtres Kubernetes	1	VM	Ubuntu 20.04.2 LTS
Nœuds workers Kubernetes	2	VM	Ubuntu 20.04.2 LTS
Nœuds workers GPU Kubernetes	2	NVIDIA DGX-1 (bare-Metal)	NVIDIA DGX OS 4.0.5 (basé sur Ubuntu 18.04.2 LTS)
Stockage	1 paire HA	Avec AFF A220	NetApp ONTAP 9.7 P6

Composant logiciel	Version
Débit d'air Apache	2.0.1
Graphique Helm à flux d'air Apache	8.0.8
Docker	19.03.12
Kubeflow	1.2
Kubernetes	1.18.9
NetApp Trident	21.01.2
NVIDIA DeepOps	Fonctionnalité de déploiement Trident depuis la succursale maître à compter de la validation "61898cdfda"; Toutes les autres fonctionnalités de la version 21.03

Assistance

NetApp ne propose pas de support pour les flux d'air Apache, Docker, Kubeflow, Kubernetes ou NVIDIA DeepOps. Si vous souhaitez une solution entièrement prise en charge avec des fonctionnalités similaires à la solution NetApp ai Control plane, "Contactez NetApp" À propos des solutions d'IA/ML entièrement prises en charge que NetApp propose conjointement avec ses partenaires.

Déploiement Kubernetes

Cette section décrit les tâches que vous devez réaliser pour déployer un cluster Kubernetes dans lequel implémenter la solution NetApp ai Control plane. Si vous disposez déjà d'un cluster Kubernetes, vous pouvez ignorer cette section tant que vous exécutez une version de Kubernetes prise en charge par Kubeflow et NetApp Trident. Pour obtenir la liste des versions Kubernetes prises en charge par Kubeflow, voir la "Documentation officielle Kubeflow". Pour obtenir la liste des versions de Kubernetes

prises en charge par Trident, consultez le "Documentation Trident".

Pour les déploiements Kubernetes sur site intégrant des nœuds bare-Metal avec processeurs graphiques NVIDIA, NetApp recommande l'utilisation de l'outil de déploiement DeepOps Kubernetes de NVIDIA. Cette section décrit le déploiement d'un cluster Kubernetes avec DeepOps.

Prérequis

Avant d'effectuer l'exercice de déploiement décrit dans cette section, nous supposons que vous avez déjà effectué les tâches suivantes :

1. Vous avez déjà configuré des nœuds Kubernetes bare-Metal (par exemple, un système NVIDIA DGX qui fait partie d'un pod ONTAP ai) conformément aux instructions de configuration standard.
2. Vous avez installé un système d'exploitation pris en charge sur tous les nœuds maîtres et workers Kubernetes et sur un hôte de démarrage du déploiement. Pour obtenir la liste des systèmes d'exploitation pris en charge par DeepOps, reportez-vous au "Site GitHub DeepOps".

Utilisez NVIDIA DeepOps pour installer et configurer Kubernetes

Pour déployer et configurer votre cluster Kubernetes avec NVIDIA DeepOps, effectuez les tâches suivantes à partir d'un hôte de démarrage du déploiement :

1. Téléchargez NVIDIA DeepOps en suivant les instructions sur le "Page de démarrage" Sur le site GitHub NVIDIA DeepOps.
2. Déployez Kubernetes dans votre cluster en suivant les instructions du "Page Guide de déploiement Kubernetes" Sur le site GitHub NVIDIA DeepOps.

Déploiement et configuration de NetApp Trident

Déploiement et configuration de NetApp Trident

Cette section décrit les tâches que vous devez effectuer pour installer et configurer NetApp Trident dans votre cluster Kubernetes.

Prérequis

Avant d'effectuer l'exercice de déploiement décrit dans cette section, nous supposons que vous avez déjà effectué les tâches suivantes :

1. Vous disposez déjà d'un cluster Kubernetes opérationnel, et vous exécutez une version de Kubernetes prise en charge par Trident. Pour obtenir la liste des versions prises en charge, reportez-vous à la section "Documentation Trident".
2. Vous disposez déjà d'un dispositif de stockage NetApp, d'une instance Software-defined ou d'un service de stockage cloud, pris en charge par Trident.

Installation de Trident

Pour installer et configurer NetApp Trident dans votre cluster Kubernetes, effectuez les tâches suivantes à partir de l'hôte de démarrage du déploiement :

1. Déployez Trident selon l'une des méthodes suivantes :

- Si vous avez utilisé NVIDIA DeepOps pour déployer votre cluster Kubernetes, vous pouvez également utiliser NVIDIA DeepOps pour déployer Trident dans votre cluster Kubernetes. Pour déployer Trident avec DeepOps, suivez le "Instructions de déploiement de Trident" Sur le site GitHub NVIDIA DeepOps.
- Si vous n'avez pas utilisé NVIDIA DeepOps pour déployer votre cluster Kubernetes ou si vous préférez simplement déployer Trident manuellement, vous pouvez déployer Trident en suivant les instructions du "instructions de déploiement" Dans la documentation Trident. Veillez à créer au moins un système back-end Trident et au moins une classe de stockage Kubernetes pour plus d'informations sur la configuration "Systèmes back-end" et "Les classes de stockage" Consultez les sous-sections liées dans NetApp Docs.

> Si vous déployez la solution NetApp ai Control plane sur un pod ONTAP ai, consultez la "Exemple de systèmes back-end Trident pour les déploiements d'IA de ONTAP" Pour certains exemples de systèmes back-end Trident, vous pouvez créer et "Exemple de Kubernetes Storageclasse pour les déploiements d'IA ONTAP" Vous voudrez peut-être créer des exemples de classes de stockage Kubernetes différentes.

Exemple de systèmes back-end Trident pour les déploiements d'IA de ONTAP

Avant de pouvoir utiliser Trident pour provisionner dynamiquement les ressources de stockage dans votre cluster Kubernetes, vous devez créer un ou plusieurs systèmes back-end Trident. Les exemples suivants représentent différents types de systèmes back-end que vous pouvez créer si vous déployez la solution NetApp ai Control plane sur un pod ONTAP ai. Pour plus d'informations sur les systèmes back-end, reportez-vous au "Documentation Trident".

1. NetApp recommande de créer un système back-end Trident compatible avec FlexGroup pour chaque LIF de données (interface réseau logique offrant un accès aux données) que vous souhaitez utiliser sur votre système NetApp AFF. Vous pourrez ainsi équilibrer les montages de volumes entre les LIF

   Les exemples de commandes ci-dessous montrent la création de deux backends Trident compatibles avec FlexGroup pour deux LIF de données différentes associées à un même SVM (Storage Virtual machine) ONTAP. Ces extrémités arrière utilisent le `ontap-nas-flexgroup` pilote de stockage ONTAP prend en charge deux principaux types de volumes de données : FlexVol et FlexGroup. La taille des volumes FlexVol est limitée (à compter de cette écriture, la taille maximale dépend du déploiement spécifique). Au contraire, les volumes FlexGroup peuvent évoluer de manière linéaire jusqu'à 20 po et 400 milliards de fichiers, fournissant un espace de nom unique qui simplifie considérablement la gestion des données. Par conséquent, les volumes FlexGroup sont optimaux pour les workloads d'IA et DE ML qui s'appuient sur des quantités importantes de données.

   Si vous travaillez avec un volume de données réduit et que vous souhaitez utiliser des volumes FlexVol plutôt que des volumes FlexGroup, vous pouvez créer des systèmes back-end Trident qui utilisent le `ontap-nas` pilote de stockage au lieu du `ontap-nas-flexgroup` pilote de stockage

```
$ cat << EOF > ./trident-backend-ontap-ai-flexgroups-iface1.json
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "backendName": "ontap-ai-flexgroups-iface1",
    "managementLIF": "10.61.218.100",
```

```
    "dataLIF": "192.168.11.11",
    "svm": "ontapai_nfs",
    "username": "admin",
    "password": "ontapai"
}
EOF
$ tridentctl create backend -f ./trident-backend-ontap-ai-flexgroups-
iface1.json -n trident
+----------------------------+---------------------
+--------------------------------------+--------+---------+
|            NAME            |   STORAGE DRIVER    |
UUID                 | STATE  | VOLUMES |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
| ontap-ai-flexgroups-iface1 | ontap-nas-flexgroup | b74cbddb-e0b8-40b7-
b263-b6da6dec0bdd | online |       0 |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
$ cat << EOF > ./trident-backend-ontap-ai-flexgroups-iface2.json
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "backendName": "ontap-ai-flexgroups-iface2",
    "managementLIF": "10.61.218.100",
    "dataLIF": "192.168.12.12",
    "svm": "ontapai_nfs",
    "username": "admin",
    "password": "ontapai"
}
EOF
$ tridentctl create backend -f ./trident-backend-ontap-ai-flexgroups-
iface2.json -n trident
+----------------------------+---------------------
+--------------------------------------+--------+---------+
|            NAME            |   STORAGE DRIVER    |
UUID                 | STATE  | VOLUMES |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
| ontap-ai-flexgroups-iface2 | ontap-nas-flexgroup | 61814d48-c770-436b-
9cb4-cf7ee661274d | online |       0 |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
$ tridentctl get backend -n trident
+----------------------------+---------------------
+--------------------------------------+--------+---------+
|            NAME            |   STORAGE DRIVER    |
```

```
UUID                  | STATE   | VOLUMES |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
| ontap-ai-flexgroups-iface1 | ontap-nas-flexgroup | b74cbddb-e0b8-40b7-
b263-b6da6dec0bdd | online |       0 |
| ontap-ai-flexgroups-iface2 | ontap-nas-flexgroup | 61814d48-c770-436b-
9cb4-cf7ee661274d | online |       0 |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
```

2. NetApp recommande également de créer un ou plusieurs systèmes back-end Trident compatibles avec FlexVol. Si vous utilisez les volumes FlexGroup pour le stockage des jeux de données d'entraînement, vous pouvez utiliser les volumes FlexVol pour stocker les résultats, les résultats, les informations de débogage, etc. Si vous souhaitez utiliser des volumes FlexVol, vous devez créer un ou plusieurs systèmes back-end Trident compatibles avec FlexVol. Les exemples de commandes ci-dessous montrent la création d'un système back-end Trident compatible avec FlexVol utilisant un seul LIF de données.

```
$ cat << EOF > ./trident-backend-ontap-ai-flexvols.json
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "ontap-ai-flexvols",
    "managementLIF": "10.61.218.100",
    "dataLIF": "192.168.11.11",
    "svm": "ontapai_nfs",
    "username": "admin",
    "password": "ontapai"
}
EOF
$ tridentctl create backend -f ./trident-backend-ontap-ai-flexvols.json -n
trident
+----------------------------+---------------------
+--------------------------------------+--------+---------+
|            NAME            |   STORAGE DRIVER    |                 UUID
| STATE  | VOLUMES |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
| ontap-ai-flexvols          | ontap-nas           | 52bdb3b1-13a5-4513-
a9c1-52a69657fabe | online |       0 |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
$ tridentctl get backend -n trident
+----------------------------+---------------------
+--------------------------------------+--------+---------+
|            NAME            |   STORAGE DRIVER    |                 UUID
| STATE  | VOLUMES |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
| ontap-ai-flexvols          | ontap-nas           | 52bdb3b1-13a5-4513-
a9c1-52a69657fabe | online |       0 |
| ontap-ai-flexgroups-iface1 | ontap-nas-flexgroup | b74cbddb-e0b8-40b7-
b263-b6da6dec0bdd | online |       0 |
| ontap-ai-flexgroups-iface2 | ontap-nas-flexgroup | 61814d48-c770-436b-
9cb4-cf7ee661274d | online |       0 |
+----------------------------+---------------------
+--------------------------------------+--------+---------+
```

Exemple de classes de stockage Kubernetes pour les déploiements d'IA ONTAP

Avant de pouvoir utiliser Trident pour provisionner les ressources de stockage de façon dynamique dans votre cluster Kubernetes, vous devez créer une ou plusieurs classes de stockage Kubernetes. Les exemples ci-dessous représentent les différents types de

classes de stockage que vous pourriez souhaiter créer si vous déployez la solution NetApp ai Control plane sur un pod ONTAP ai. Pour plus d'informations sur les classes de stockage, consultez le "Documentation Trident".

1. NetApp recommande de créer une classe de stockage distincte pour chaque système back-end Trident compatible avec FlexGroup que vous avez créé dans la section "Exemple de systèmes back-end Trident pour les déploiements d'IA de ONTAP", étape 1. Ces classes de stockage granulaire vous permettent d'ajouter des montages NFS qui correspondent à des LIF spécifiques (les LIF que vous avez spécifiées lors de la création des backends Trident) en tant que système back-end spécifique spécifié dans le fichier des spécifications de classe de stockage. Les exemples de commandes qui suivent montrent la création de deux classes de stockage qui correspondent aux deux exemples de backends créés dans la section "Exemple de systèmes back-end Trident pour les déploiements d'IA de ONTAP", étape 1. Pour plus d'informations sur les classes de stockage, consultez le "Documentation Trident".

   Pour qu'un volume persistant ne soit pas supprimé lorsque la demande de volume persistant correspondante est supprimée, l'exemple suivant utilise une `reclaimPolicy` valeur de `Retain`. Pour plus d'informations sur le `reclaimPolicy` consultez le champ officiel "Documentation Kubernetes".

```
$ cat << EOF > ./storage-class-ontap-ai-flexgroups-retain-iface1.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontap-ai-flexgroups-retain-iface1
provisioner: netapp.io/trident
parameters:
  backendType: "ontap-nas-flexgroup"
  storagePools: "ontap-ai-flexgroups-iface1:.*"
reclaimPolicy: Retain
EOF
$ kubectl create -f ./storage-class-ontap-ai-flexgroups-retain-
iface1.yaml
storageclass.storage.k8s.io/ontap-ai-flexgroups-retain-iface1 created
$ cat << EOF > ./storage-class-ontap-ai-flexgroups-retain-iface2.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontap-ai-flexgroups-retain-iface2
provisioner: netapp.io/trident
parameters:
  backendType: "ontap-nas-flexgroup"
  storagePools: "ontap-ai-flexgroups-iface2:.*"
reclaimPolicy: Retain
EOF
$ kubectl create -f ./storage-class-ontap-ai-flexgroups-retain-
iface2.yaml
storageclass.storage.k8s.io/ontap-ai-flexgroups-retain-iface2 created
$ kubectl get storageclass
NAME                                PROVISIONER         AGE
ontap-ai-flexgroups-retain-iface1   netapp.io/trident   0m
ontap-ai-flexgroups-retain-iface2   netapp.io/trident   0m
```

2. NetApp recommande également de créer une classe de stockage correspondant au système back-end Trident activé pour FlexVol que vous avez créé dans la section "Exemple de systèmes back-end Trident pour les déploiements d'IA de ONTAP", étape 2. Les exemples de commandes ci-dessous montrent la création d'une classe de stockage unique pour les volumes FlexVol.

   Dans l'exemple suivant, un système back-end particulier n'est pas spécifié dans le fichier de définition StorageClass car un seul système back-end Trident activé pour FlexVol a été créé. Lorsque vous utilisez Kubernetes pour administrer des volumes qui utilisent cette classe de stockage, Trident tente d'utiliser tout back-end disponible qui utilise le `ontap-nas` conducteur.

```
$ cat << EOF > ./storage-class-ontap-ai-flexvols-retain.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontap-ai-flexvols-retain
provisioner: netapp.io/trident
parameters:
  backendType: "ontap-nas"
reclaimPolicy: Retain
EOF
$ kubectl create -f ./storage-class-ontap-ai-flexvols-retain.yaml
storageclass.storage.k8s.io/ontap-ai-flexvols-retain created
$ kubectl get storageclass
NAME                                PROVISIONER         AGE
ontap-ai-flexgroups-retain-iface1   netapp.io/trident   1m
ontap-ai-flexgroups-retain-iface2   netapp.io/trident   1m
ontap-ai-flexvols-retain            netapp.io/trident   0m
```

3. NetApp recommande également de créer une classe de stockage générique pour les volumes FlexGroup. Les exemples de commandes suivants montrent la création d'une classe de stockage générique pour les volumes FlexGroup.

   Notez qu'un back-end particulier n'est pas spécifié dans le fichier de définition de classe de stockage. Par conséquent, lorsque vous utilisez Kubernetes pour gérer des volumes qui utilisent cette classe de stockage, Trident tente d'utiliser tout back-end disponible qui utilise le `ontap-nas-flexgroup` conducteur.

```
$ cat << EOF > ./storage-class-ontap-ai-flexgroups-retain.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontap-ai-flexgroups-retain
provisioner: netapp.io/trident
parameters:
  backendType: "ontap-nas-flexgroup"
reclaimPolicy: Retain
EOF
$ kubectl create -f ./storage-class-ontap-ai-flexgroups-retain.yaml
storageclass.storage.k8s.io/ontap-ai-flexgroups-retain created
$ kubectl get storageclass
NAME                                PROVISIONER         AGE
ontap-ai-flexgroups-retain          netapp.io/trident   0m
ontap-ai-flexgroups-retain-iface1   netapp.io/trident   2m
ontap-ai-flexgroups-retain-iface2   netapp.io/trident   2m
ontap-ai-flexvols-retain            netapp.io/trident   1m
```

Déploiement Kubeflow

Cette section décrit les tâches à effectuer pour déployer Kubeflow dans votre cluster Kubernetes.

Prérequis

Avant d'effectuer l'exercice de déploiement décrit dans cette section, nous supposons que vous avez déjà effectué les tâches suivantes :

1. Vous disposez déjà d'un cluster Kubernetes opérationnel, et vous exécutez une version de Kubernetes prise en charge par Kubeflow. Pour obtenir la liste des versions prises en charge, reportez-vous à la section "Documentation officielle Kubeflow".
2. Vous avez déjà installé et configuré NetApp Trident dans votre cluster Kubernetes, comme indiqué dans la "Déploiement et configuration de Trident".

Définissez la classe de stockage Kubernetes par défaut

Avant de déployer Kubeflow, vous devez désigner une classe de stockage par défaut dans votre cluster Kubernetes. Le processus de déploiement Kubeflow tente de provisionner de nouveaux volumes persistants à l'aide de la classe de stockage par défaut. Si aucune classe de stockage n'est désignée comme classe de stockage par défaut, le déploiement échoue. Pour désigner une classe de stockage par défaut dans votre cluster, effectuez la tâche suivante à partir de l'hôte de démarrage du déploiement. Si vous avez déjà désigné une classe de stockage par défaut dans votre cluster, vous pouvez ignorer cette étape.

1. Désignez une des classes de stockage existantes comme classe de stockage par défaut. Les exemples de commandes ci-dessous montrent la désignation d'une classe de stockage nommée `ontap-ai-flexvols-retain` Comme classe de stockage par défaut.

Le `ontap-nas-flexgroup` Le type de volume interne Trident a une taille de volume persistant minimale et est relativement élevée. Par défaut, Kubeflow tente de provisionner des demandes de volume qui n'ont que quelques Go. Par conséquent, vous ne devez pas désigner une classe de stockage utilisant le `ontap-nas-flexgroup` Taper back-end comme classe de stockage par défaut pour le déploiement Kubeflow.

```
$ kubectl get sc
NAME                                  PROVISIONER             AGE
ontap-ai-flexgroups-retain            csi.trident.netapp.io   25h
ontap-ai-flexgroups-retain-iface1     csi.trident.netapp.io   25h
ontap-ai-flexgroups-retain-iface2     csi.trident.netapp.io   25h
ontap-ai-flexvols-retain              csi.trident.netapp.io   3s
$ kubectl patch storageclass ontap-ai-flexvols-retain -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
storageclass.storage.k8s.io/ontap-ai-flexvols-retain patched
$ kubectl get sc
NAME                                   PROVISIONER             AGE
ontap-ai-flexgroups-retain             csi.trident.netapp.io   25h
ontap-ai-flexgroups-retain-iface1      csi.trident.netapp.io   25h
ontap-ai-flexgroups-retain-iface2      csi.trident.netapp.io   25h
ontap-ai-flexvols-retain (default)     csi.trident.netapp.io   54s
```

Utilisez NVIDIA DeepOps pour déployer Kubeflow

NetApp recommande d'utiliser l'outil de déploiement Kubeflow qui est fourni par NVIDIA DeepOps. Pour déployer Kubeflow dans votre cluster Kubernetes à l'aide de l'outil de déploiement DeepOps, effectuez les tâches suivantes à partir de l'hôte de démarrage du déploiement.

Vous pouvez également déployer Kubeflow manuellement en suivant les options "instructions d'installation" Dans la documentation officielle Kubeflow

1. Déployez Kubeflow dans votre cluster en suivant les options "Instructions de déploiement Kubeflow" Sur le site GitHub NVIDIA DeepOps.
2. Notez l'URL du tableau de bord Kubeflow que fournit l'outil de déploiement DeepOps Kubeflow.

```
$ ./scripts/k8s/deploy_kubeflow.sh -x
…
INFO[0007] Applied the configuration Successfully!
filename="cmd/apply.go:72"
Kubeflow app installed to: /home/ai/kubeflow
It may take several minutes for all services to start. Run 'kubectl get
pods -n kubeflow' to verify
To remove (excluding CRDs, istio, auth, and cert-manager), run:
./scripts/k8s_deploy_kubeflow.sh -d
To perform a full uninstall : ./scripts/k8s_deploy_kubeflow.sh -D
Kubeflow Dashboard (HTTP NodePort): http://10.61.188.111:31380
```

3. Vérifier que tous les pods déployés dans l'espace de noms Kubeflow affichent un `STATUS` de `Running` et assurez-vous qu'aucun composant déployé dans le namespace n'est à l'état d'erreur. Le démarrage des modules peut prendre plusieurs minutes.

```
$ kubectl get all -n kubeflow
NAME                                                              READY
STATUS    RESTARTS   AGE
pod/admission-webhook-bootstrap-stateful-set-0                    1/1
Running   0          95s
pod/admission-webhook-deployment-6b89c84c98-vrtbh                 1/1
Running   0          91s
pod/application-controller-stateful-set-0                         1/1
Running   0          98s
pod/argo-ui-5dcf5d8b4f-m2wn4                                      1/1
Running   0          97s
pod/centraldashboard-cf4874ddc-7hcr8                              1/1
Running   0          97s
pod/jupyter-web-app-deployment-685b455447-gjhh7                   1/1
Running   0          96s
pod/katib-controller-88c97d85c-kgq66                              1/1
Running   1          95s
pod/katib-db-8598468fd8-5jw2c                                     1/1
Running   0          95s
pod/katib-manager-574c8c67f9-wtrf5                                1/1
Running   1          95s
pod/katib-manager-rest-778857c989-fjbzn                           1/1
Running   0          95s
pod/katib-suggestion-bayesianoptimization-65df4d7455-qthmw        1/1
Running   0          94s
pod/katib-suggestion-grid-56bf69f597-98vwn                        1/1
Running   0          94s
pod/katib-suggestion-hyperband-7777b76cb9-9v6dq                   1/1
Running   0          93s
```

```
pod/katib-suggestion-nasrl-77f6f9458c-2qzxq                    1/1
Running   0          93s
pod/katib-suggestion-random-77b88b5c79-l64j9                   1/1
Running   0          93s
pod/katib-ui-7587c5b967-nd629                                  1/1
Running   0          95s
pod/metacontroller-0                                           1/1
Running   0          96s
pod/metadata-db-5dd459cc-swzkm                                 1/1
Running   0          94s
pod/metadata-deployment-6cf77db994-69fk7                       1/1
Running   3          93s
pod/metadata-deployment-6cf77db994-mpbjt                       1/1
Running   3          93s
pod/metadata-deployment-6cf77db994-xg7tz                       1/1
Running   3          94s
pod/metadata-ui-78f5b59b56-qb6kr                               1/1
Running   0          94s
pod/minio-758b769d67-llvdr                                     1/1
Running   0          91s
pod/ml-pipeline-5875b9db95-g8t2k                               1/1
Running   0          91s
pod/ml-pipeline-persistenceagent-9b69ddd46-bt9r9               1/1
Running   0          90s
pod/ml-pipeline-scheduledworkflow-7b8d756c76-7x56s             1/1
Running   0          90s
pod/ml-pipeline-ui-79ffd9c76-fcwpd                             1/1
Running   0          90s
pod/ml-pipeline-viewer-controller-deployment-5fdc87f58-b2t9r   1/1
Running   0          90s
pod/mysql-657f87857d-l5k9z                                     1/1
Running   0          91s
pod/notebook-controller-deployment-56b4f59bbf-8bvnr            1/1
Running   0          92s
pod/profiles-deployment-6bc745947-mrdkh                        2/2
Running   0          90s
pod/pytorch-operator-77c97f4879-hmlrv                          1/1
Running   0          92s
pod/seldon-operator-controller-manager-0                       1/1
Running   1          91s
pod/spartakus-volunteer-5fdfddb779-l7qkm                       1/1
Running   0          92s
pod/tensorboard-6544748d94-nh8b2                               1/1
Running   0          92s
pod/tf-job-dashboard-56f79c59dd-6w59t                          1/1
Running   0          92s
```

```
pod/tf-job-operator-79cbfd6dbc-rb58c                          1/1
Running   0          91s
pod/workflow-controller-db644d554-cwrnb                       1/1
Running   0          97s
NAME                                                     TYPE
CLUSTER-IP      EXTERNAL-IP   PORT(S)             AGE
service/admission-webhook-service                        ClusterIP
10.233.51.169   <none>        443/TCP             97s
service/application-controller-service                   ClusterIP
10.233.4.54     <none>        443/TCP             98s
service/argo-ui                                          NodePort
10.233.47.191   <none>        80:31799/TCP        97s
service/centraldashboard                                 ClusterIP
10.233.8.36     <none>        80/TCP              97s
service/jupyter-web-app-service                          ClusterIP
10.233.1.42     <none>        80/TCP              97s
service/katib-controller                                 ClusterIP
10.233.25.226   <none>        443/TCP             96s
service/katib-db                                         ClusterIP
10.233.33.151   <none>        3306/TCP            97s
service/katib-manager                                    ClusterIP
10.233.46.239   <none>        6789/TCP            96s
service/katib-manager-rest                               ClusterIP
10.233.55.32    <none>        80/TCP              96s
service/katib-suggestion-bayesianoptimization            ClusterIP
10.233.49.191   <none>        6789/TCP            95s
service/katib-suggestion-grid                            ClusterIP
10.233.9.105    <none>        6789/TCP            95s
service/katib-suggestion-hyperband                       ClusterIP
10.233.22.2     <none>        6789/TCP            95s
service/katib-suggestion-nasrl                           ClusterIP
10.233.63.73    <none>        6789/TCP            95s
service/katib-suggestion-random                          ClusterIP
10.233.57.210   <none>        6789/TCP            95s
service/katib-ui                                         ClusterIP
10.233.6.116    <none>        80/TCP              96s
service/metadata-db                                      ClusterIP
10.233.31.2     <none>        3306/TCP            96s
service/metadata-service                                 ClusterIP
10.233.27.104   <none>        8080/TCP            96s
service/metadata-ui                                      ClusterIP
10.233.57.177   <none>        80/TCP              96s
service/minio-service                                    ClusterIP
10.233.44.90    <none>        9000/TCP            94s
service/ml-pipeline                                      ClusterIP
10.233.41.201   <none>        8888/TCP,8887/TCP   94s
```

```
service/ml-pipeline-tensorboard-ui                   ClusterIP
10.233.36.207   <none>        80/TCP              93s
service/ml-pipeline-ui                               ClusterIP
10.233.61.150   <none>        80/TCP              93s
service/mysql                                        ClusterIP
10.233.55.117   <none>        3306/TCP            94s
service/notebook-controller-service                  ClusterIP
10.233.10.166   <none>        443/TCP             95s
service/profiles-kfam                                ClusterIP
10.233.33.79    <none>        8081/TCP            92s
service/pytorch-operator                             ClusterIP
10.233.37.112   <none>        8443/TCP            95s
service/seldon-operator-controller-manager-service   ClusterIP
10.233.30.178   <none>        443/TCP             92s
service/tensorboard                                  ClusterIP
10.233.58.151   <none>        9000/TCP            94s
service/tf-job-dashboard                             ClusterIP
10.233.4.17     <none>        80/TCP              94s
service/tf-job-operator                              ClusterIP
10.233.60.32    <none>        8443/TCP            94s
service/webhook-server-service                       ClusterIP
10.233.32.167   <none>        443/TCP             87s
NAME                                                      READY   UP-
TO-DATE   AVAILABLE   AGE
deployment.apps/admission-webhook-deployment              1/1     1
1           97s
deployment.apps/argo-ui                                   1/1     1
1           97s
deployment.apps/centraldashboard                          1/1     1
1           97s
deployment.apps/jupyter-web-app-deployment                1/1     1
1           97s
deployment.apps/katib-controller                          1/1     1
1           96s
deployment.apps/katib-db                                  1/1     1
1           97s
deployment.apps/katib-manager                             1/1     1
1           96s
deployment.apps/katib-manager-rest                        1/1     1
1           96s
deployment.apps/katib-suggestion-bayesianoptimization     1/1     1
1           95s
deployment.apps/katib-suggestion-grid                     1/1     1
1           95s
deployment.apps/katib-suggestion-hyperband                1/1     1
1           95s
```

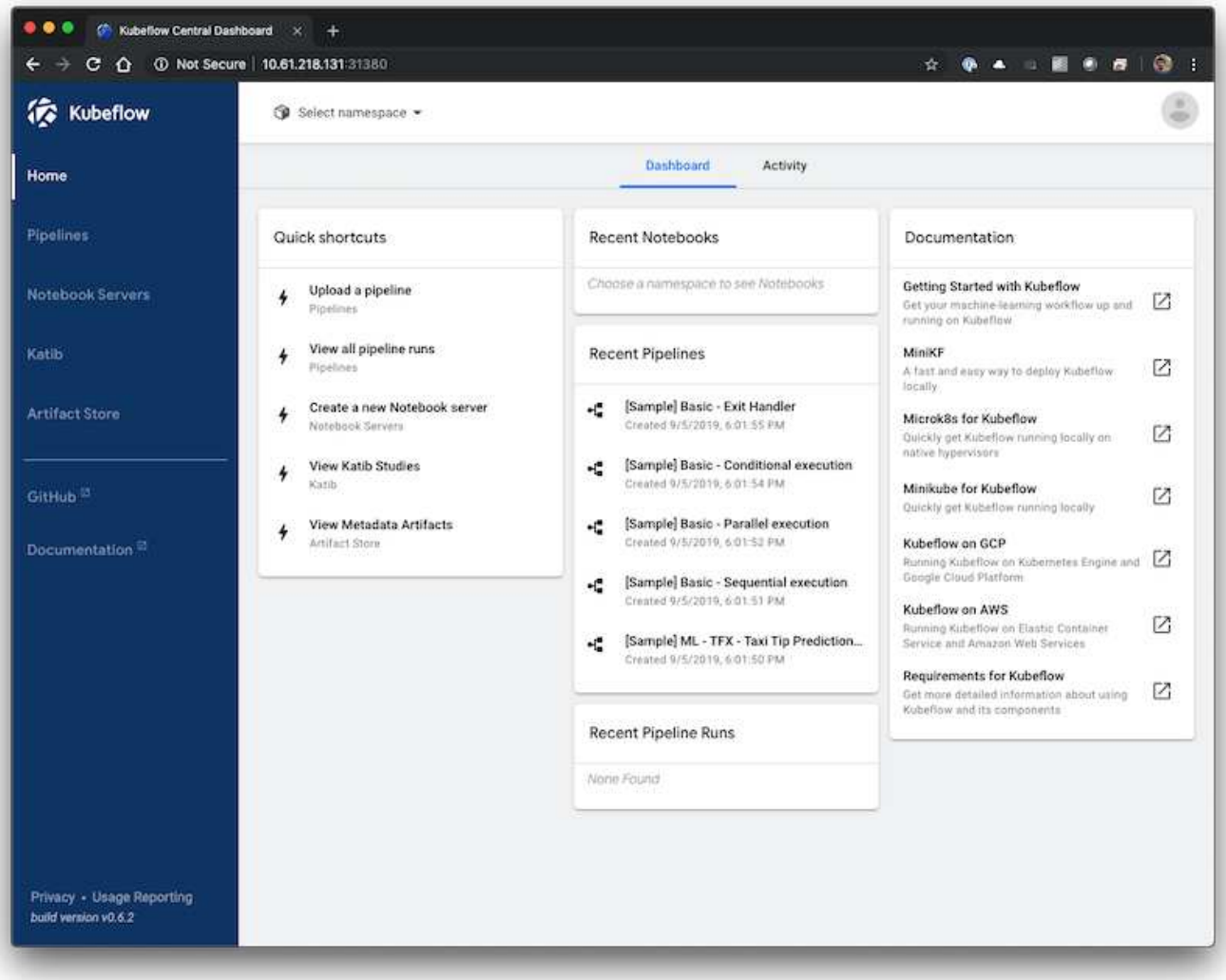
```
deployment.apps/katib-suggestion-nasrl                     1/1     1
1           95s
deployment.apps/katib-suggestion-random                    1/1     1
1           95s
deployment.apps/katib-ui                                   1/1     1
1           96s
deployment.apps/metadata-db                                1/1     1
1           96s
deployment.apps/metadata-deployment                        3/3     3
3           96s
deployment.apps/metadata-ui                                1/1     1
1           96s
deployment.apps/minio                                      1/1     1
1           94s
deployment.apps/ml-pipeline                                1/1     1
1           94s
deployment.apps/ml-pipeline-persistenceagent               1/1     1
1           93s
deployment.apps/ml-pipeline-scheduledworkflow              1/1     1
1           93s
deployment.apps/ml-pipeline-ui                             1/1     1
1           93s
deployment.apps/ml-pipeline-viewer-controller-deployment   1/1     1
1           93s
deployment.apps/mysql                                      1/1     1
1           94s
deployment.apps/notebook-controller-deployment             1/1     1
1           95s
deployment.apps/profiles-deployment                        1/1     1
1           92s
deployment.apps/pytorch-operator                           1/1     1
1           95s
deployment.apps/spartakus-volunteer                        1/1     1
1           94s
deployment.apps/tensorboard                                1/1     1
1           94s
deployment.apps/tf-job-dashboard                           1/1     1
1           94s
deployment.apps/tf-job-operator                            1/1     1
1           94s
deployment.apps/workflow-controller                        1/1     1
1           97s
NAME
DESIRED   CURRENT   READY   AGE
replicaset.apps/admission-webhook-deployment-6b89c84c98              1
1         1       97s
```

```
replicaset.apps/argo-ui-5dcf5d8b4f                                  1
1         1       97s
replicaset.apps/centraldashboard-cf4874ddc                          1
1         1       97s
replicaset.apps/jupyter-web-app-deployment-685b455447               1
1         1       97s
replicaset.apps/katib-controller-88c97d85c                          1
1         1       96s
replicaset.apps/katib-db-8598468fd8                                 1
1         1       97s
replicaset.apps/katib-manager-574c8c67f9                            1
1         1       96s
replicaset.apps/katib-manager-rest-778857c989                       1
1         1       96s
replicaset.apps/katib-suggestion-bayesianoptimization-65df4d7455    1
1         1       95s
replicaset.apps/katib-suggestion-grid-56bf69f597                    1
1         1       95s
replicaset.apps/katib-suggestion-hyperband-7777b76cb9               1
1         1       95s
replicaset.apps/katib-suggestion-nasrl-77f6f9458c                   1
1         1       95s
replicaset.apps/katib-suggestion-random-77b88b5c79                  1
1         1       95s
replicaset.apps/katib-ui-7587c5b967                                 1
1         1       96s
replicaset.apps/metadata-db-5dd459cc                                1
1         1       96s
replicaset.apps/metadata-deployment-6cf77db994                      3
3         3       96s
replicaset.apps/metadata-ui-78f5b59b56                              1
1         1       96s
replicaset.apps/minio-758b769d67                                    1
1         1       93s
replicaset.apps/ml-pipeline-5875b9db95                              1
1         1       93s
replicaset.apps/ml-pipeline-persistenceagent-9b69ddd46              1
1         1       92s
replicaset.apps/ml-pipeline-scheduledworkflow-7b8d756c76            1
1         1       91s
replicaset.apps/ml-pipeline-ui-79ffd9c76                            1
1         1       91s
replicaset.apps/ml-pipeline-viewer-controller-deployment-5fdc87f58  1
1         1       91s
replicaset.apps/mysql-657f87857d                                    1
1         1       92s
```

```
replicaset.apps/notebook-controller-deployment-56b4f59bbf            1
1         1       94s
replicaset.apps/profiles-deployment-6bc745947                        1
1         1       91s
replicaset.apps/pytorch-operator-77c97f4879                          1
1         1       94s
replicaset.apps/spartakus-volunteer-5fdfddb779                       1
1         1       94s
replicaset.apps/tensorboard-6544748d94                               1
1         1       93s
replicaset.apps/tf-job-dashboard-56f79c59dd                          1
1         1       93s
replicaset.apps/tf-job-operator-79cbfd6dbc                           1
1         1       93s
replicaset.apps/workflow-controller-db644d554                        1
1         1       97s
NAME                                                        READY   AGE
statefulset.apps/admission-webhook-bootstrap-stateful-set   1/1     97s
statefulset.apps/application-controller-stateful-set        1/1     98s
statefulset.apps/metacontroller                             1/1     98s
statefulset.apps/seldon-operator-controller-manager         1/1     92s
$ kubectl get pvc -n kubeflow
NAME             STATUS   VOLUME
CAPACITY   ACCESS MODES   STORAGECLASS               AGE
katib-mysql      Bound    pvc-b07f293e-d028-11e9-9b9d-00505681a82d
10Gi       RWO            ontap-ai-flexvols-retain   27m
metadata-mysql   Bound    pvc-b0f3f032-d028-11e9-9b9d-00505681a82d
10Gi       RWO            ontap-ai-flexvols-retain   27m
minio-pv-claim   Bound    pvc-b22727ee-d028-11e9-9b9d-00505681a82d
20Gi       RWO            ontap-ai-flexvols-retain   27m
mysql-pv-claim   Bound    pvc-b2429afd-d028-11e9-9b9d-00505681a82d
20Gi       RWO            ontap-ai-flexvols-retain   27m
```

4. Dans votre navigateur web, accédez au tableau de bord central Kubeflow en accédant à l'URL que vous avez notée à l'étape 2.

   Le nom d'utilisateur par défaut est `admin@kubeflow.org`, et le mot de passe par défaut est `12341234`. Pour créer d'autres utilisateurs, suivez les instructions de la section "Documentation officielle Kubeflow".



Exemple d’opérations et de tâches Kubeflow

Cette section fournit des exemples de diverses opérations et tâches que vous pouvez exécuter avec Kubeflow.

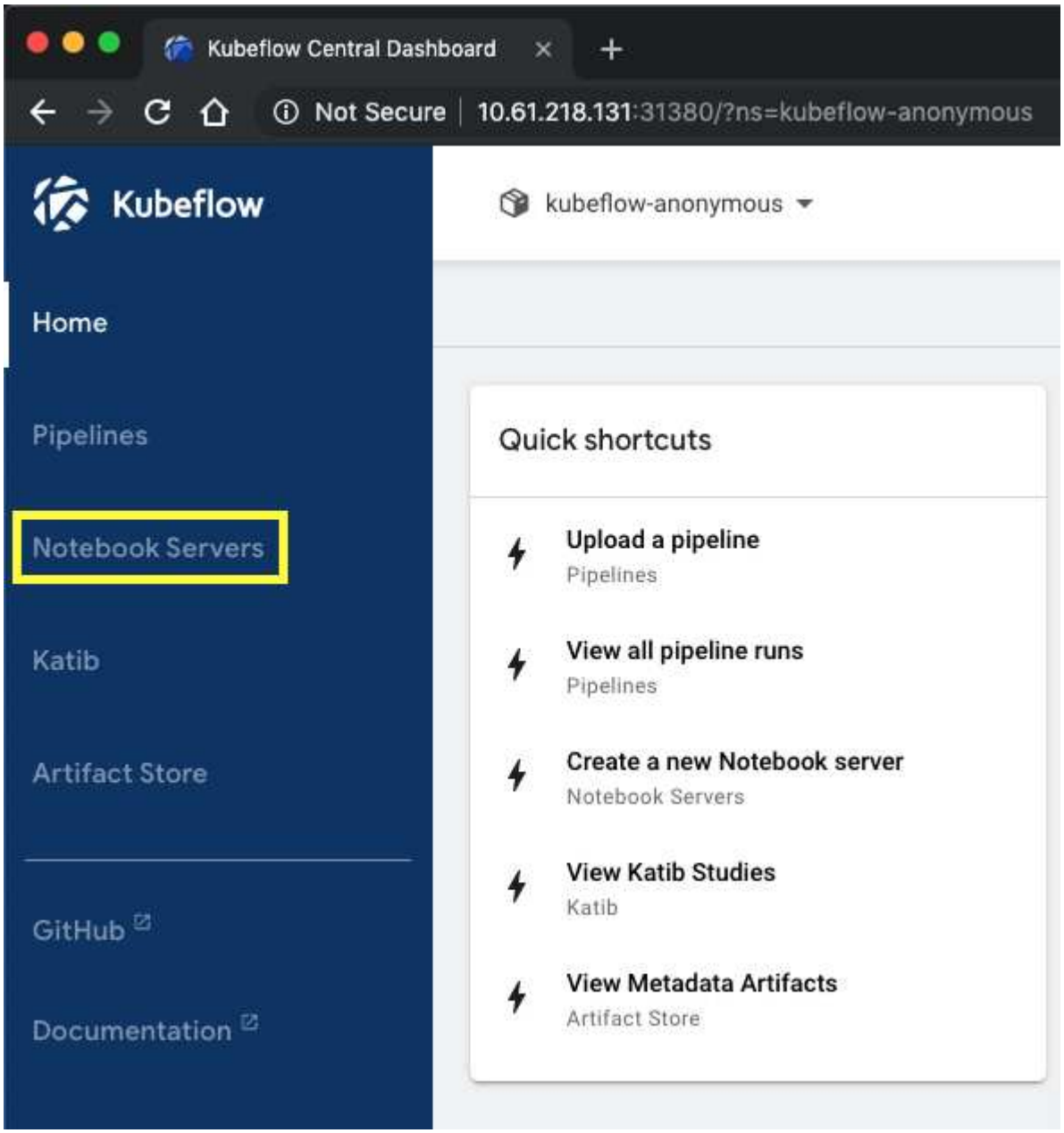
Exemple d’opérations et de tâches Kubeflow

Cette section fournit des exemples de diverses opérations et tâches que vous pouvez exécuter avec Kubeflow.

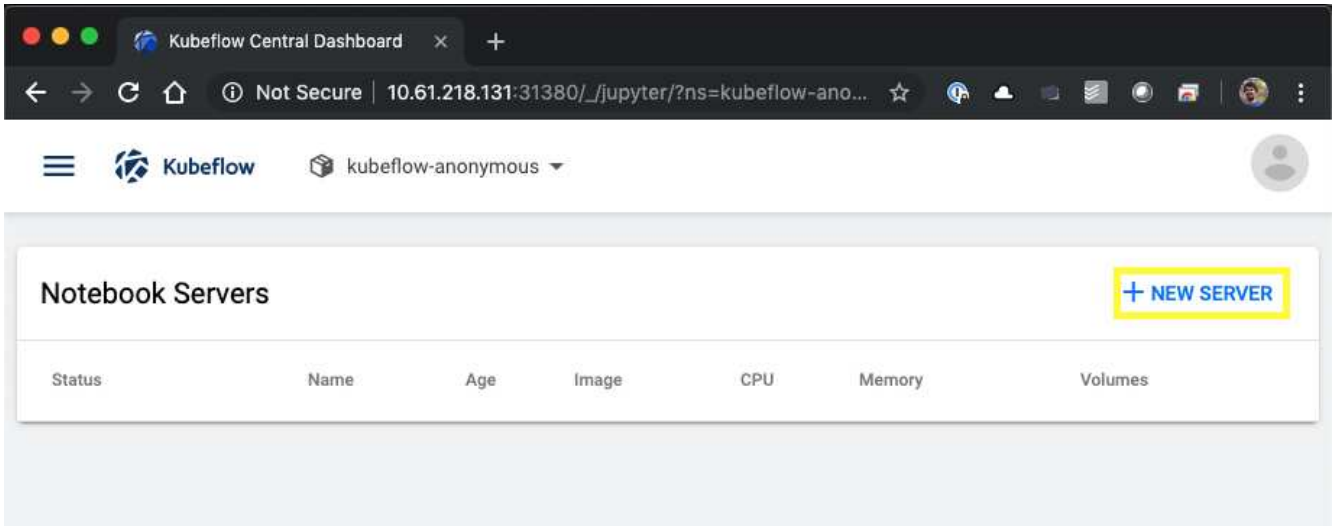
Provisionner un espace de travail Jupyter Notebook pour un usage Data Scientist ou Developer

Kubeflow est capable de provisionner rapidement de nouveaux serveurs Jupyter Notebook pour agir en tant qu’espaces de travail de data Scientist. Pour provisionner un nouveau serveur Jupyter Notebook avec Kubeflow, effectuez les tâches suivantes. Pour plus d’informations sur les ordinateurs portables Jupyter dans le contexte Kubeflow, reportez-vous au "Documentation officielle Kubeflow".

1. Dans le tableau de bord central Kubeflow, cliquez sur serveurs portables dans le menu principal pour accéder à la page d'administration des serveurs Jupyter Notebook.

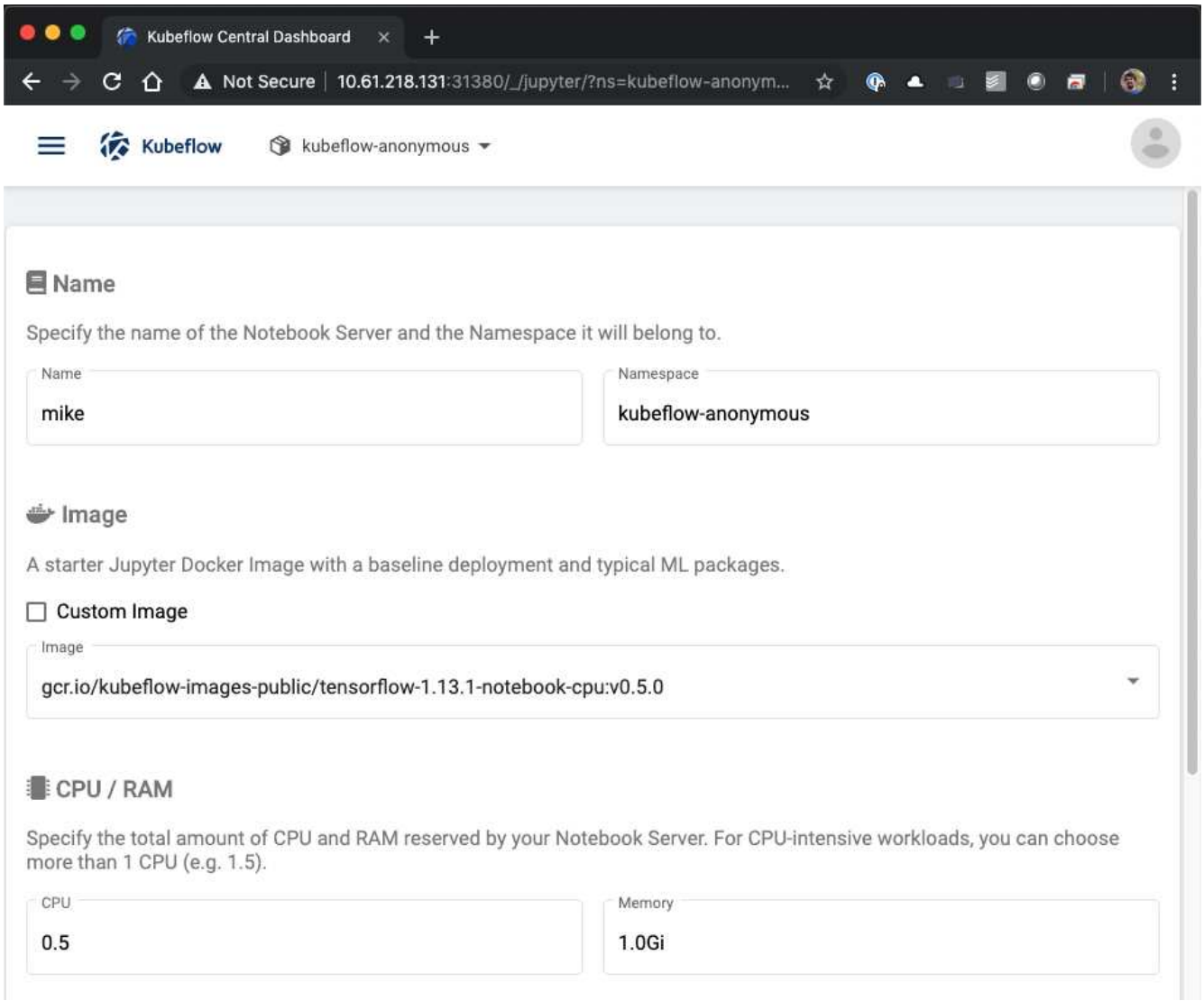


2. Cliquez sur Nouveau serveur pour provisionner un nouveau serveur Jupyter Notebook.

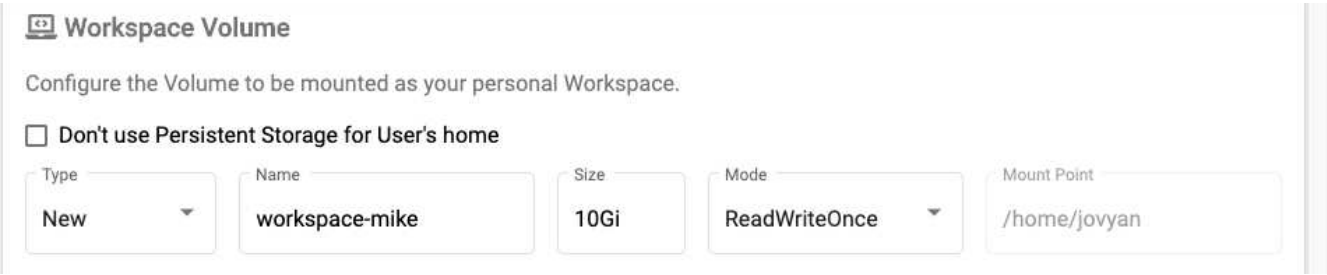


3. Donnez un nom à votre nouveau serveur, choisissez l'image Docker sur laquelle vous voulez que votre serveur soit basé, et spécifiez la quantité de CPU et de RAM à réserver par votre serveur. Si le champ espace de noms est vide, utilisez le menu Sélectionner espace de noms dans l'en-tête de la page pour choisir un espace de noms. Le champ espace de noms est alors automatiquement renseigné avec l'espace de noms choisi.

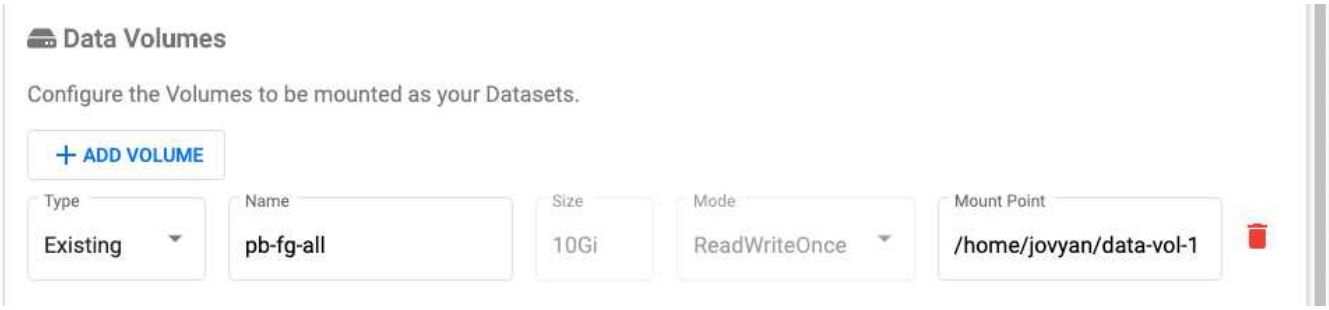
   Dans l'exemple suivant, le `kubeflow-anonymous` l'espace de noms est choisi. En outre, les valeurs par défaut pour l'image Docker, la CPU et la RAM sont acceptées.



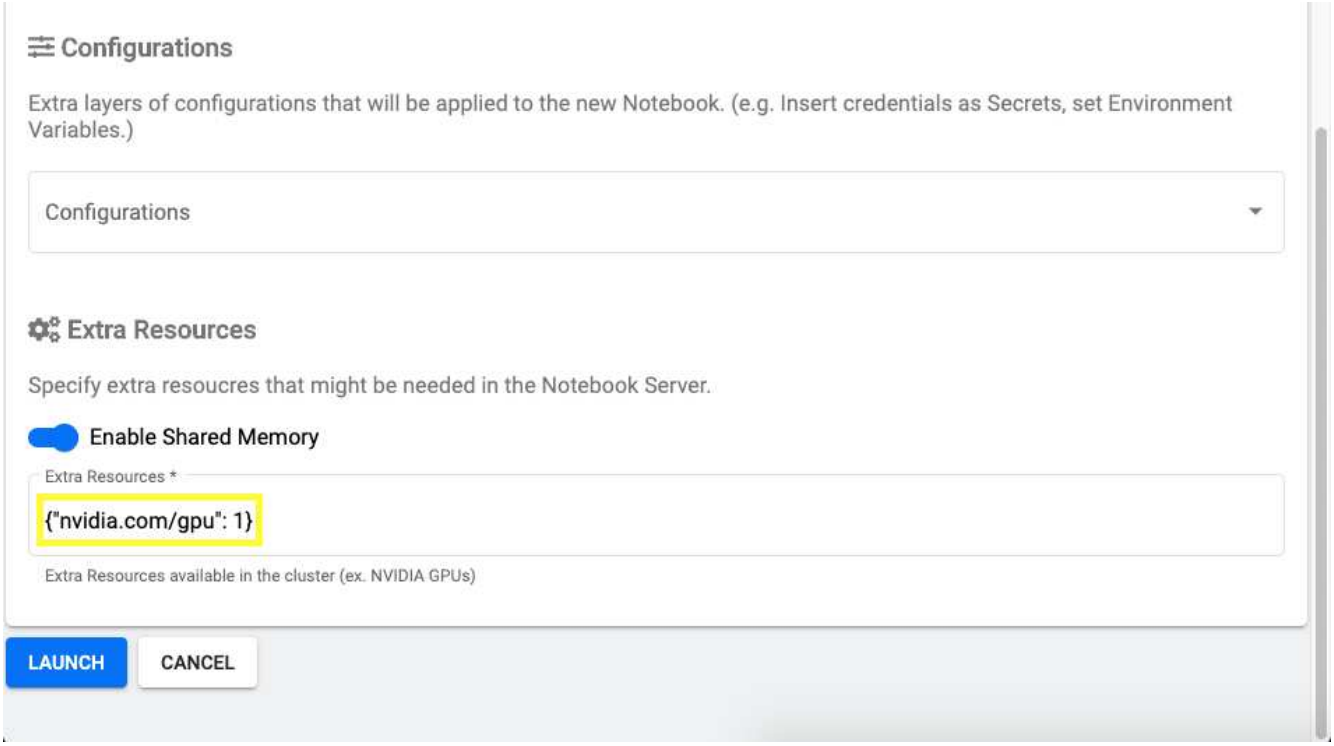
4. Spécifiez les détails du volume de l'espace de travail. Si vous choisissez de créer un nouveau volume, ce volume ou cette demande de volume persistant est provisionné à l'aide de la classe de stockage par défaut. Comme une classe de stockage utilisant Trident a été désignée comme classe de stockage par défaut dans la section "Déploiement Kubeflow", Le volume ou la demande de volume persistant est provisionné avec Trident. Ce volume est automatiquement monté comme espace de travail par défaut dans le conteneur Jupyter Notebook Server. Tous les ordinateurs portables créés par un utilisateur sur le serveur qui ne sont pas enregistrés dans un volume de données distinct sont automatiquement enregistrés dans ce volume d'espace de travail. Par conséquent, les ordinateurs portables sont persistants entre les redémarrages.



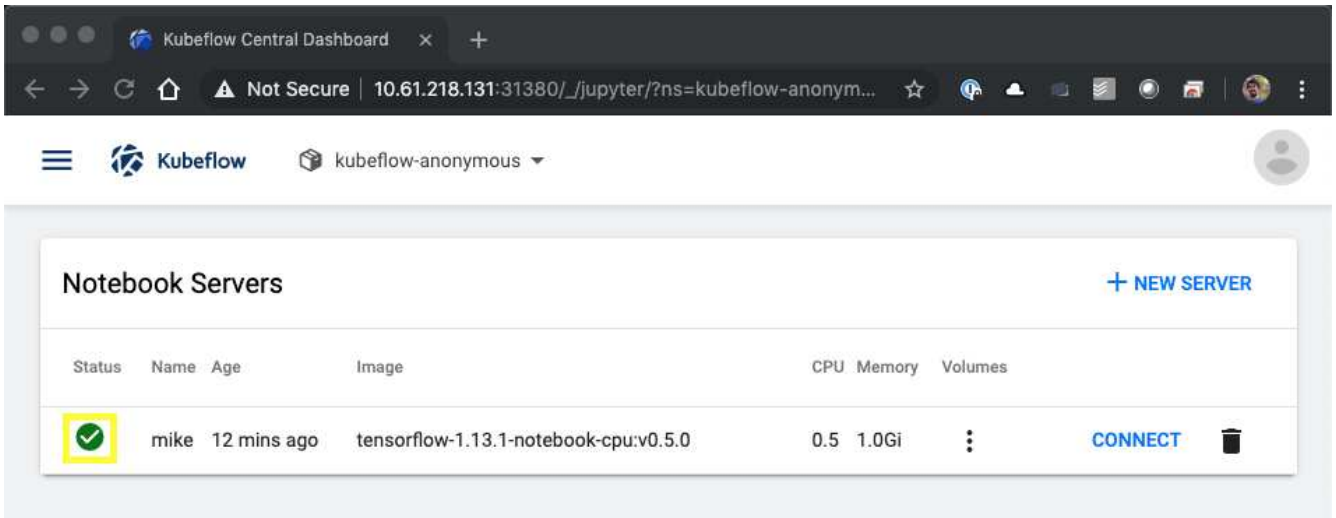
5. Ajout de volumes de données. L'exemple suivant indique un PVC existant nommé 'pb-fg-all' et accepte le point de montage par défaut.



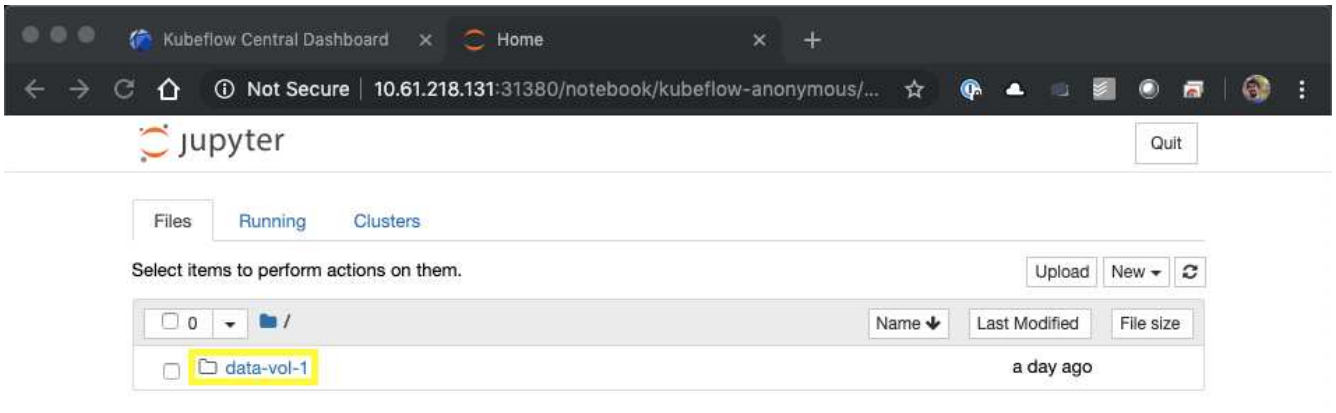
6. **Facultatif:** Demandez que le nombre de processeurs graphiques souhaité soit alloué à votre serveur d'ordinateur portable. Dans l'exemple suivant, un GPU est demandé.

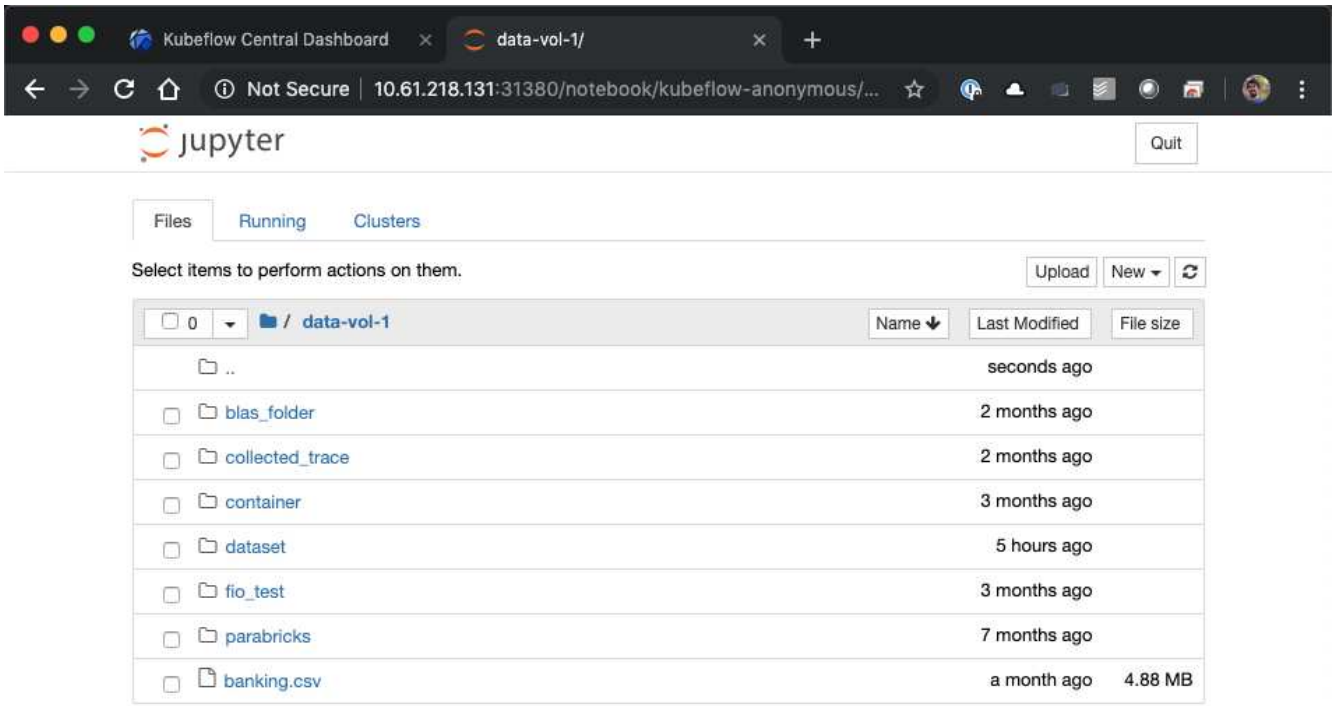


7. Cliquez sur lancer pour approvisionner votre nouveau serveur d'ordinateur portable.
8. Attendez que le serveur de votre ordinateur portable soit entièrement approvisionné. Cette opération peut prendre plusieurs minutes si vous n'avez jamais provisionné de serveur à l'aide de l'image Docker que vous avez spécifiée, car l'image doit être téléchargée. Lorsque votre serveur a été entièrement provisionné, une coche verte s'affiche dans la colonne État de la page d'administration du serveur Jupyter Notebook.



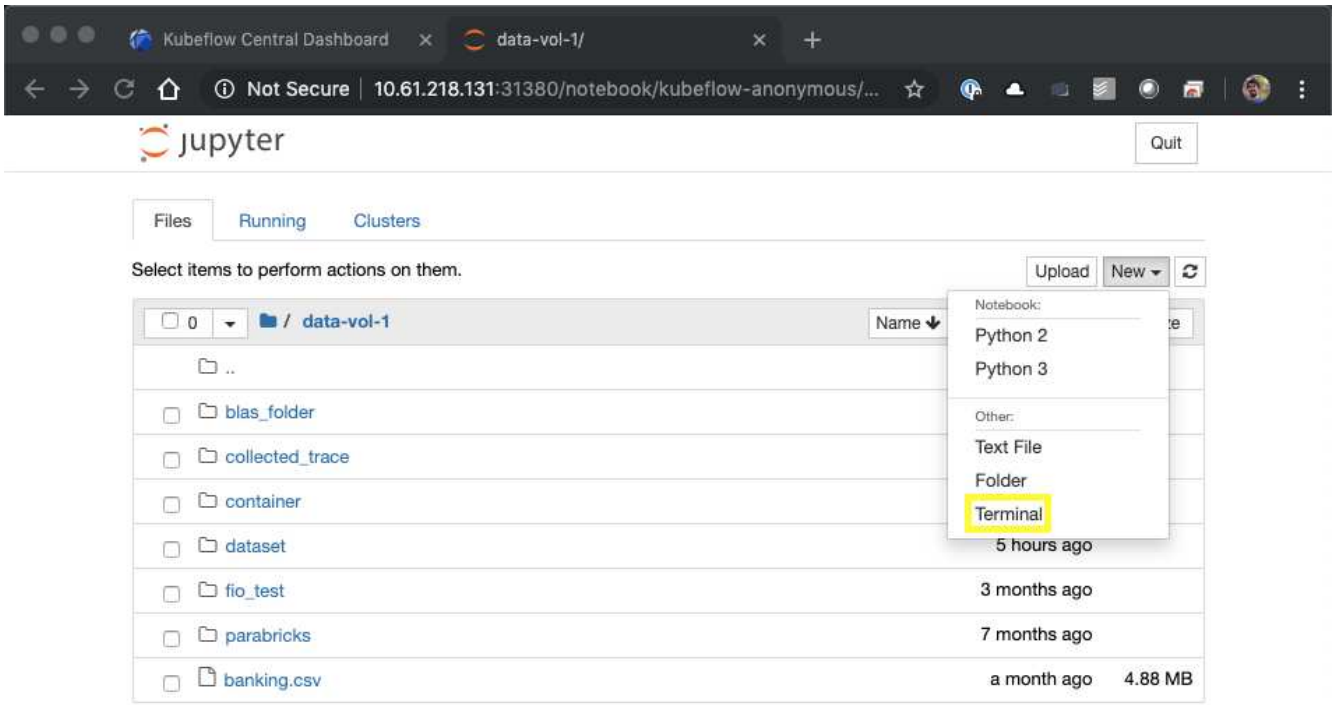
9. Cliquez sur connexion pour vous connecter à votre nouvelle interface Web de serveur.
10. Vérifiez que le volume du dataset spécifié à l'étape 6 est monté sur le serveur. Notez que ce volume est monté par défaut dans l'espace de travail par défaut. Du point de vue de l'utilisateur, ce n'est qu'un autre dossier dans l'espace de travail. L'utilisateur, qui est probablement un spécialiste des données et non un expert en infrastructure, n'a pas besoin de posséder de compétences en stockage pour utiliser ce volume.





11. Ouvrez un terminal et, en supposant qu'un nouveau volume a été demandé à l'étape 5, exécutez `df -h` Pour confirmer qu'un nouveau volume persistant provisionné avec Trident est monté en tant qu'espace de travail par défaut.

    Le répertoire d'espace de travail par défaut est le répertoire de base avec lequel vous êtes présenté lorsque vous accédez pour la première fois à l'interface Web du serveur. Par conséquent, tout artefact que vous créez via l'interface Web est stocké sur ce volume persistant provisionné par Trident.



```
$ df -h
Filesystem                                                      Size  Used Avail
Use% Mounted on
overlay                                                         439G   34G  382G
  9% /
tmpfs                                                            64M     0   64M
  0% /dev
tmpfs                                                           252G     0  252G
  0% /sys/fs/cgroup
/dev/sda2                                                       439G   34G  382G
  9% /etc/hosts
192.168.11.11:/trident_pvc_3dcfe7e5_d5a9_11e9_9b9d_00505681a82d   10G  320K   10G
  1% /home/jovyan
tmpfs                                                           252G     0  252G
  0% /dev/shm
192.168.11.11:/pb_fg_all                                         10T   10T   47G
100% /home/jovyan/data-vol-1
tmpfs                                                           252G   12K  252G
  1% /run/secrets/kubernetes.io/serviceaccount
tmpfs                                                           252G   12K  252G
  1% /proc/driver/nvidia
tmpfs                                                            51G  4.9M   51G
  1% /run/nvidia-persistenced/socket
udev                                                            252G     0  252G
  0% /dev/nvidia5
tmpfs                                                           252G     0  252G
  0% /proc/acpi
tmpfs                                                           252G     0  252G
  0% /proc/scsi
tmpfs                                                           252G     0  252G
  0% /sys/firmware
$
```

12. À l'aide du terminal, exécutez `nvidia-smi` Pour vérifier que le nombre correct de GPU a été attribué au serveur d'ordinateur portable. Dans l'exemple suivant, un GPU a été affecté au serveur d'ordinateurs portables comme demandé à l'étape 7.

```
$ nvidia-smi
Fri Sep 13 13:52:15 2019
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 410.104      Driver Version: 410.104      CUDA Version: N/A      |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|===============================+======================+======================|
|   0  Tesla V100-SXM2...  On   | 00000000:86:00.0 Off |                    0 |
| N/A   38C    P0    46W / 300W |      0MiB / 32480MiB |      0%      Default |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                       GPU Memory |
|  GPU       PID   Type   Process name                             Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
$
```

Exemple de carnets de bord et de pipelines

Le "Kit NetApp Data Science Toolkit pour Kubernetes" Peut être utilisé avec Kubeflow. Grâce au kit NetApp Data Science Toolkit avec Kubeflow, il offre les avantages suivants :

- Les data Scientists peuvent effectuer des opérations avancées de gestion des données NetApp directement depuis un ordinateur portable Jupyter.
- Les opérations avancées de gestion des données NetApp peuvent être intégrées aux workflows automatisés grâce à la structure Kubeflow pipelines.

Reportez-vous à la "Exemples Kubeflow" Section du référentiel GitHub pour le kit NetApp Data Science pour plus d'informations sur l'utilisation du kit avec Kubeflow.

Déploiement du flux d'air Apache

NetApp recommande d'exécuter le flux d'air Apache sur Kubernetes. Cette section décrit les tâches à effectuer pour déployer l'air dans votre cluster Kubernetes.

> Il est possible de déployer du flux d'air sur d'autres plateformes que Kubernetes. Le déploiement de flux d'air sur des plateformes autres que Kubernetes ne fait pas partie du cadre de cette solution.

Prérequis

Avant d'effectuer l'exercice de déploiement décrit dans cette section, nous supposons que vous avez déjà effectué les tâches suivantes :

1. Vous disposez déjà d'un cluster Kubernetes fonctionnel.
2. Vous avez déjà installé et configuré NetApp Trident dans votre cluster Kubernetes comme décrit dans la section « déploiement et configuration de NetApp Trident ».

Installer Helm

Le flux d'air est déployé à l'aide de Helm, un gestionnaire de packages populaire pour Kubernetes. Avant de déployer le flux d'air, vous devez installer Helm sur l'hôte de démarrage à déploiement rapide. Pour installer Helm sur l'hôte de saut de déploiement, suivez le "instructions d'installation" Dans la documentation officielle Helm.

Définissez la classe de stockage Kubernetes par défaut

Avant de déployer le flux d'air, vous devez désigner une classe de stockage par défaut dans votre cluster Kubernetes. Le processus de déploiement du flux d'air tente de provisionner de nouveaux volumes persistants à l'aide de la classe de stockage par défaut. Si aucune classe de stockage n'est désignée comme classe de stockage par défaut, le déploiement échoue. Pour désigner une classe de stockage par défaut au sein de votre cluster, suivez les instructions de la section "Déploiement Kubeflow". Si vous avez déjà désigné une classe de stockage par défaut dans votre cluster, vous pouvez ignorer cette étape.

Utilisez Helm pour déployer le flux d'air

Pour déployer le flux d'air dans votre cluster Kubernetes à l'aide de Helm, effectuez les tâches suivantes à partir de l'hôte saut de déploiement :

1. Déployer le flux d'air à l'aide de Helm en suivant le "instructions de déploiement" Pour le tableau de débit d'air officiel sur le concentrateur d'artefacts. Les exemples de commandes qui suivent montrent le déploiement du flux d'air à l'aide de Helm. Modifiez, ajoutez et/ou supprimez des valeurs dans `custom-values.yaml` fichier selon votre environnement et la configuration de votre choix.

```
$ cat << EOF > custom-values.yaml
###################################
# Airflow - Common Configs
###################################
airflow:
  ## the airflow executor type to use
  ##
  executor: "CeleryExecutor"
  ## environment variables for the web/scheduler/worker Pods (for
airflow configs)
  ##
  #
###################################
# Airflow - WebUI Configs
###################################
web:
  ## configs for the Service of the web Pods
  ##
  service:
    type: NodePort
###################################
# Airflow - Logs Configs
###################################
logs:
  persistence:
    enabled: true
###################################
# Airflow - DAGs Configs
###################################
dags:
  ## configs for the DAG git repository & sync container
  ##
  gitSync:
    enabled: true
    ## url of the git repository
    ##
    repo: "git@github.com:mboglesby/airflow-dev.git"
    ## the branch/tag/sha1 which we clone
    ##
    branch: master
    revision: HEAD
```

```
    ## the name of a pre-created secret containing files for ~/.ssh/
    ##
    ## NOTE:
    ## - this is ONLY RELEVANT for SSH git repos
    ## - the secret commonly includes files: id_rsa, id_rsa.pub,
known_hosts
    ## - known_hosts is NOT NEEDED if `git.sshKeyscan` is true
    ##
    sshSecret: "airflow-ssh-git-secret"
    ## the name of the private key file in your `git.secret`
    ##
    ## NOTE:
    ## - this is ONLY RELEVANT for PRIVATE SSH git repos
    ##
    sshSecretKey: id_rsa
    ## the git sync interval in seconds
    ##
    syncWait: 60
EOF
$ helm install airflow airflow-stable/airflow -n airflow --version 8.0.8
--values ./custom-values.yaml
...
Congratulations. You have just deployed Apache Airflow!
1. Get the Airflow Service URL by running these commands:
   export NODE_PORT=$(kubectl get --namespace airflow -o
jsonpath="{.spec.ports[0].nodePort}" services airflow-web)
   export NODE_IP=$(kubectl get nodes --namespace airflow -o
jsonpath="{.items[0].status.addresses[0].address}")
   echo http://$NODE_IP:$NODE_PORT/
2. Open Airflow in your web browser
```

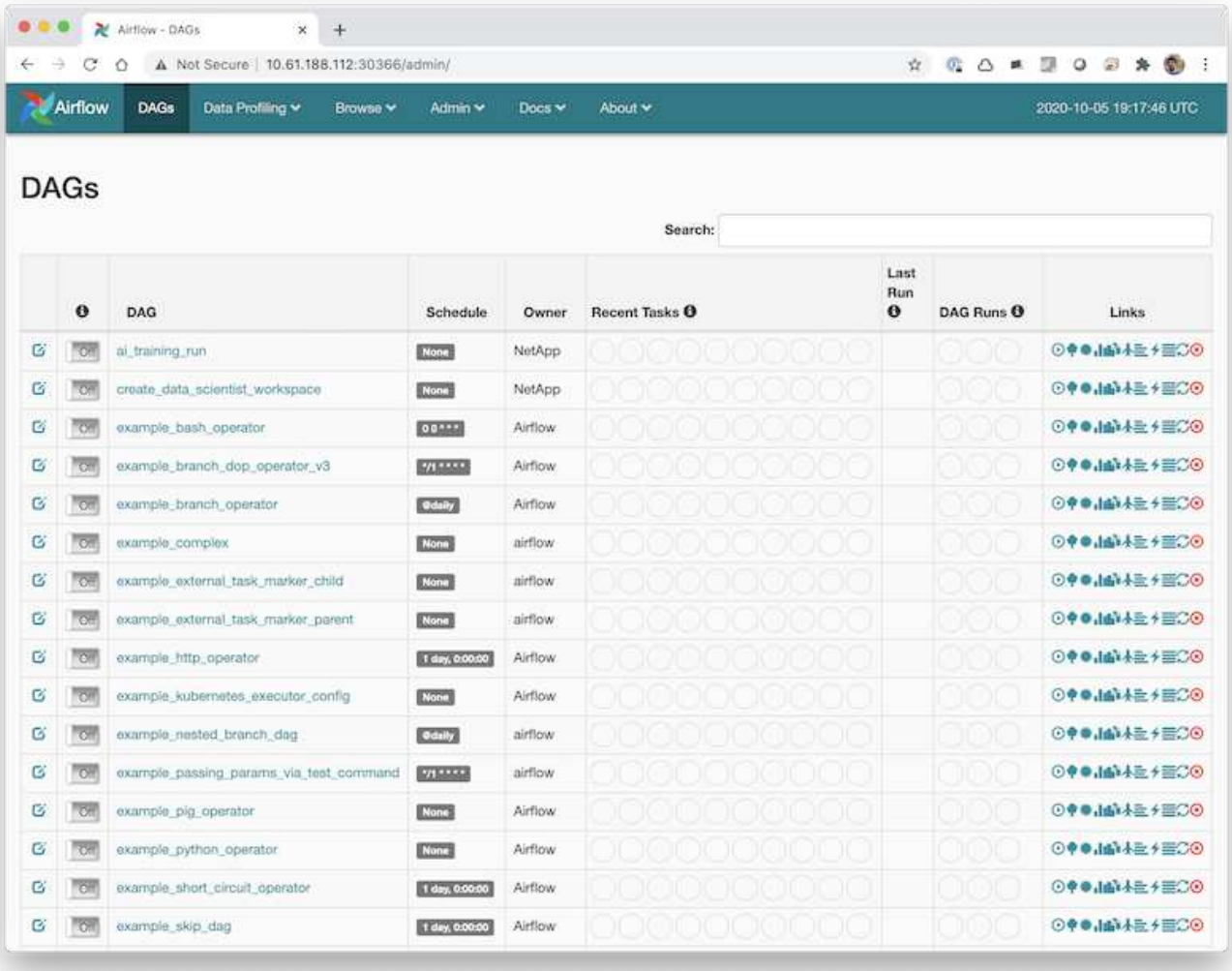
2. Assurez-vous que tous les modules de ventilation sont opérationnels. Le démarrage des modules peut prendre quelques minutes.

```
$ kubectl -n airflow get pod
NAME                                READY   STATUS    RESTARTS   AGE
airflow-flower-b5656d44f-h8qjk      1/1     Running   0          2h
airflow-postgresql-0                1/1     Running   0          2h
airflow-redis-master-0              1/1     Running   0          2h
airflow-scheduler-9d95fcdf9-clf4b   2/2     Running   2          2h
airflow-web-59c94db9c5-z7rg4        1/1     Running   0          2h
airflow-worker-0                    2/2     Running   2          2h
```

3. Pour obtenir l'URL du service Web de flux d'air, suivez les instructions qui ont été imprimées sur la console lorsque vous avez déployé du flux d'air à l'aide de Helm à l'étape 1.

```
$ export NODE_PORT=$(kubectl get --namespace airflow -o
jsonpath="{.spec.ports[0].nodePort}" services airflow-web)
$ export NODE_IP=$(kubectl get nodes --namespace airflow -o
jsonpath="{.items[0].status.addresses[0].address}")
$ echo http://$NODE_IP:$NODE_PORT/
```

4. Confirmez que vous pouvez accéder au service Web de débit d'air.



Exemple de flux de production de flux d'air Apache

Le "Kit NetApp Data Science Toolkit pour Kubernetes" Peut être utilisé en conjonction avec le débit d'air. Grâce au kit NetApp Data Science Toolkit et à la circulation de l'air, vous pouvez incorporer les opérations de gestion des données NetApp dans des workflows automatisés orchestrés par le flux de production d'air.

Reportez-vous à la "Exemples de débit d'air" Section du référentiel GitHub NetApp Data Science Toolkit pour plus d'informations sur l'utilisation du kit avec flux d'air.

Exemple d'opérations Trident

Cette section comprend des exemples d'opérations que vous pouvez effectuer avec Trident.

Importer un volume existant

Si votre système/plateforme de stockage NetApp contient des volumes que vous souhaitez monter sur des conteneurs au sein de votre cluster Kubernetes, mais qui ne sont pas liés aux demandes de volume persistant dans le cluster, vous devez importer ces volumes. Vous pouvez utiliser la fonctionnalité d'importation de volumes Trident pour importer ces volumes.

Les exemples de commandes ci-dessous montrent l'importation du même volume, nommé `pb_fg_all`, Deux fois, une fois pour chaque système back-end Trident créé dans l'exemple de la section "Exemple de systèmes back-end Trident pour les déploiements d'IA de ONTAP", étape 1. Si vous importez le même volume deux fois de cette manière, vous pouvez monter le volume (un volume FlexGroup existant) plusieurs fois sur plusieurs LIF, comme décrit dans la section "Exemple de systèmes back-end Trident pour les déploiements d'IA de ONTAP", étape 1. Pour plus d'informations sur les ESV, consultez le "Documentation officielle Kubernetes". Pour plus d'informations sur la fonctionnalité d'importation de volume, reportez-vous à la section "Documentation Trident".

An `accessModes` valeur de `ReadOnlyMany` Est spécifié dans les fichiers de spécifications PVC d'exemple. Pour plus d'informations sur le `accessMode` voir "Documentation officielle Kubernetes".

> Les noms de back-end spécifiés dans l'exemple de commandes d'importation suivant correspondent aux Backend créés dans l'exemple de la section "Exemple de systèmes back-end Trident pour les déploiements d'IA de ONTAP", étape 1. Les noms de classes de stockage spécifiés dans les exemples de fichiers de définition de PVC suivants correspondent aux classes de stockage créées dans l'exemple de la section "Exemple de classes de stockage Kubernetes pour les déploiements d'IA ONTAP", étape 1.

```
$ cat << EOF > ./pvc-import-pb_fg_all-iface1.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pb-fg-all-iface1
  namespace: default
spec:
  accessModes:
    - ReadOnlyMany
  storageClassName: ontap-ai-flexgroups-retain-iface1
EOF
$ tridentctl import volume ontap-ai-flexgroups-iface1 pb_fg_all -f ./pvc-
import-pb_fg_all-iface1.yaml -n trident
+--------------------------------+--------
+-----------------------------------+----------
+-------------------------------------------+--------+---------+
|          NAME                   |  SIZE  |       STORAGE CLASS
| PROTOCOL |             BACKEND UUID                          | STATE  |
```

```
MANAGED |
+--------------------------------+--------
+-----------------------------------+----------
+-----------------------------------------+--------+---------+
| default-pb-fg-all-iface1-7d9f1 | 10 TiB | ontap-ai-flexgroups-retain-
iface1 | file     | b74cbddb-e0b8-40b7-b263-b6da6dec0bdd | online | true
|
+--------------------------------+--------
+-----------------------------------+----------
+-----------------------------------------+--------+---------+
$ cat << EOF > ./pvc-import-pb_fg_all-iface2.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pb-fg-all-iface2
  namespace: default
spec:
  accessModes:
    - ReadOnlyMany
  storageClassName: ontap-ai-flexgroups-retain-iface2
EOF
$ tridentctl import volume ontap-ai-flexgroups-iface2 pb_fg_all -f ./pvc-
import-pb_fg_all-iface2.yaml -n trident
+--------------------------------+--------
+-----------------------------------+----------
+-----------------------------------------+--------+---------+
|          NAME                  |  SIZE  |       STORAGE CLASS
| PROTOCOL |             BACKEND UUID                        | STATE  |
MANAGED |
+--------------------------------+--------
+-----------------------------------+----------
+-----------------------------------------+--------+---------+
| default-pb-fg-all-iface2-85aee | 10 TiB | ontap-ai-flexgroups-retain-
iface2 | file     | 61814d48-c770-436b-9cb4-cf7ee661274d | online | true
|
+--------------------------------+--------
+-----------------------------------+----------
+-----------------------------------------+--------+---------+
$ tridentctl get volume -n trident
+-----------------------------------+---------
+-----------------------------------+----------
+-----------------------------------+--------+---------+
|               NAME                |  SIZE   |           STORAGE CLASS
| PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+-----------------------------------+---------
+-----------------------------------+----------
```

```
+---------------------------------------+--------+---------+
| default-pb-fg-all-iface1-7d9f1   | 10 TiB  | ontap-ai-flexgroups-retain-
iface1 | file     | b74cbddb-e0b8-40b7-b263-b6da6dec0bdd | online | true
|
| default-pb-fg-all-iface2-85aee   | 10 TiB  | ontap-ai-flexgroups-retain-
iface2 | file     | 61814d48-c770-436b-9cb4-cf7ee661274d | online | true
|
+----------------------------------+---------
+-----------------------------------+----------
+---------------------------------------+--------+---------+
$ kubectl get pvc
NAME                 STATUS   VOLUME                                   CAPACITY
ACCESS MODES   STORAGECLASS                        AGE
pb-fg-all-iface1     Bound    default-pb-fg-all-iface1-7d9f1
10995116277760   ROX            ontap-ai-flexgroups-retain-iface1   25h
pb-fg-all-iface2     Bound    default-pb-fg-all-iface2-85aee
10995116277760   ROX            ontap-ai-flexgroups-retain-iface2   25h
```

Provisionner un nouveau volume

Vous pouvez utiliser Trident pour provisionner un nouveau volume sur votre système ou plateforme de stockage NetApp. L'exemple suivant montre le provisionnement d'un nouveau volume FlexVol. Dans cet exemple, le volume est provisionné à l'aide de la classe de stockage créée dans l'exemple de la section "Exemple de classes de stockage Kubernetes pour les déploiements d'IA ONTAP", étape 2.

An `accessModes` valeur de `ReadWriteMany` Est spécifié dans l'exemple de fichier de définition de PVC suivant. Pour plus d'informations sur le `accessMode` voir "Documentation officielle Kubernetes".

```
$ cat << EOF > ./pvc-tensorflow-results.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: tensorflow-results
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
  storageClassName: ontap-ai-flexvols-retain
EOF
$ kubectl create -f ./pvc-tensorflow-results.yaml
persistentvolumeclaim/tensorflow-results created
$ kubectl get pvc
NAME                              STATUS    VOLUME
CAPACITY         ACCESS MODES   STORAGECLASS                        AGE
pb-fg-all-iface1                  Bound     default-pb-fg-all-iface1-7d9f1
10995116277760   ROX            ontap-ai-flexgroups-retain-iface1   26h
pb-fg-all-iface2                  Bound     default-pb-fg-all-iface2-85aee
10995116277760   ROX            ontap-ai-flexgroups-retain-iface2   26h
tensorflow-results                Bound     default-tensorflow-results-
2fd60   1073741824       RWX            ontap-ai-flexvols-retain
25h
```

Exemple d’tâches hautes performances pour les déploiements d’IA ONTAP

Cette section comprend des exemples de tâches haute performance pouvant être exécutées lorsque Kubernetes est déployé sur un pod ONTAP ai.

Exemple d’tâches hautes performances pour les déploiements d’IA ONTAP

Cette section comprend des exemples de tâches haute performance pouvant être exécutées lorsque Kubernetes est déployé sur un pod ONTAP ai.

Exécutez un workload d’IA à un seul nœud

Pour exécuter une tâche d’IA et DE ML à un seul nœud dans votre cluster Kubernetes, effectuez les tâches suivantes à partir de l’hôte de démarrage du déploiement. Trident vous permet de faire rapidement et facilement un volume de données, potentiellement contenant des pétaoctets de données, accessibles pour un workload Kubernetes. Pour rendre ce volume de données accessible depuis un pod Kubernetes, il vous suffit de

spécifier une demande de volume persistant dans la définition du pod. Cette étape est une opération Kubernetes native ; aucune expertise de NetApp n'est requise.

> Dans cette section, vous devez déjà avoir conteneurisé (au format du conteneur Docker) le workload d'IA et DE ML spécifique que vous essayez d'exécuter dans votre cluster Kubernetes.

1. L'exemple de commandes suivant montre la création d'un travail Kubernetes pour un workload de banc d'essai TensorFlow qui utilise le dataset ImageNet. Pour plus d'informations sur le dataset ImageNet, reportez-vous à la "Site Web ImageNET".

   Cet exemple de tâche nécessite huit GPU, soit un nœud worker doté d'au moins huit GPU. Cet exemple de tâche peut être envoyée dans un cluster pour lequel un nœud worker doté d'au moins huit GPU n'est pas présent ou est actuellement occupé par une autre charge de travail. Si c'est le cas, le travail reste à l'état en attente jusqu'à ce qu'un nœud de travail soit disponible.

   En outre, pour optimiser la bande passante de stockage, le volume contenant les données d'entraînement requises est monté deux fois dans le pod que cette tâche crée. Un autre volume est également monté dans le pod. Ce deuxième volume sera utilisé pour stocker les résultats et les mesures. Ces volumes sont référencés dans la définition de travail en utilisant les noms des ESV. Pour plus d'informations sur les tâches Kubernetes, consultez le "Documentation officielle Kubernetes".

   An `emptyDir` volume avec un `medium` valeur de `Memory` est monté sur `/dev/shm` dans le pod créé par cet exemple de travail. La taille par défaut du `/dev/shm` Le volume virtuel créé automatiquement par le runtime des conteneurs Docker peut parfois manquer pour TensorFlow. Montage d'un `emptyDir` comme dans l'exemple suivant, le volume est suffisamment grand `/dev/shm` volume virtuel. Pour plus d'informations sur `emptyDir` les volumes, voir "Documentation officielle Kubernetes".

   Le conteneur unique spécifié dans cet exemple de définition de travail est donné un `securityContext > privileged` valeur de `true`. Cette valeur signifie que le conteneur dispose d'un accès racine sur l'hôte. Cette annotation est utilisée dans ce cas, car la charge de travail spécifique exécutée nécessite un accès racine. Plus précisément, une opération de mise en cache claire exécutée par la charge de travail nécessite un accès racine. Si cela est ou non `privileged: true` l'annotation est nécessaire dépend des exigences de la charge de travail spécifique que vous exécutez.

```
$ cat << EOF > ./netapp-tensorflow-single-imagenet.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: netapp-tensorflow-single-imagenet
spec:
  backoffLimit: 5
  template:
    spec:
      volumes:
      - name: dshm
        emptyDir:
          medium: Memory
      - name: testdata-iface1
        persistentVolumeClaim:
          claimName: pb-fg-all-iface1
```

```
      - name: testdata-iface2
        persistentVolumeClaim:
          claimName: pb-fg-all-iface2
      - name: results
        persistentVolumeClaim:
          claimName: tensorflow-results
      containers:
      - name: netapp-tensorflow-py2
        image: netapp/tensorflow-py2:19.03.0
        command: ["python", "/netapp/scripts/run.py", "--
dataset_dir=/mnt/mount_0/dataset/imagenet", "--dgx_version=dgx1", "--
num_devices=8"]
        resources:
          limits:
            nvidia.com/gpu: 8
        volumeMounts:
        - mountPath: /dev/shm
          name: dshm
        - mountPath: /mnt/mount_0
          name: testdata-iface1
        - mountPath: /mnt/mount_1
          name: testdata-iface2
        - mountPath: /tmp
          name: results
        securityContext:
          privileged: true
      restartPolicy: Never
EOF
$ kubectl create -f ./netapp-tensorflow-single-imagenet.yaml
job.batch/netapp-tensorflow-single-imagenet created
$ kubectl get jobs
NAME                                       COMPLETIONS   DURATION   AGE
netapp-tensorflow-single-imagenet          0/1           24s        24s
```

2. Vérifiez que le travail que vous avez créé à l'étape 1 fonctionne correctement. L'exemple de commande suivant confirme qu'un seul pod a été créé pour le travail, comme spécifié dans la définition du travail, et que ce pod s'exécute actuellement sur l'un des nœuds workers GPU.

```
$ kubectl get pods -o wide
NAME                                             READY   STATUS
RESTARTS   AGE
IP              NODE            NOMINATED NODE
netapp-tensorflow-single-imagenet-m7x92          1/1     Running     0
3m    10.233.68.61    10.61.218.154   <none>
```

3. Vérifiez que le travail que vous avez créé à l'étape 1 s'est terminé avec succès. L'exemple de commandes suivant confirme que le travail a été terminé avec succès.

```
$ kubectl get jobs
NAME                                          COMPLETIONS   DURATION
AGE
netapp-tensorflow-single-imagenet             1/1           5m42s
10m
$ kubectl get pods
NAME                                                  READY   STATUS
RESTARTS   AGE
netapp-tensorflow-single-imagenet-m7x92               0/1     Completed
0          11m
$ kubectl logs netapp-tensorflow-single-imagenet-m7x92
[netapp-tensorflow-single-imagenet-m7x92:00008] PMIX ERROR: NO-
PERMISSIONS in file gds_dstore.c at line 702
[netapp-tensorflow-single-imagenet-m7x92:00008] PMIX ERROR: NO-
PERMISSIONS in file gds_dstore.c at line 711
Total images/sec = 6530.59125
================ Clean Cache !!! ==================
mpirun -allow-run-as-root -np 1 -H localhost:1 bash -c 'sync; echo 1 >
/proc/sys/vm/drop_caches'
==========================================
mpirun -allow-run-as-root -np 8 -H localhost:8 -bind-to none -map-by
slot -x NCCL_DEBUG=INFO -x LD_LIBRARY_PATH -x PATH python
/netapp/tensorflow/benchmarks_190205/scripts/tf_cnn_benchmarks/tf_cnn_be
nchmarks.py --model=resnet50 --batch_size=256 --device=gpu
--force_gpu_compatible=True --num_intra_threads=1 --num_inter_threads=48
--variable_update=horovod --batch_group_size=20 --num_batches=500
--nodistortions --num_gpus=1 --data_format=NCHW --use_fp16=True
--use_tf_layers=False --data_name=imagenet --use_datasets=True
--data_dir=/mnt/mount_0/dataset/imagenet
--datasets_parallel_interleave_cycle_length=10
--datasets_sloppy_parallel_interleave=False --num_mounts=2
--mount_prefix=/mnt/mount_%d --datasets_prefetch_buffer_size=2000
--datasets_use_prefetch=True --datasets_num_private_threads=4
--horovod_device=gpu >
/tmp/20190814_105450_tensorflow_horovod_rdma_resnet50_gpu_8_256_b500_ima
genet_nodistort_fp16_r10_m2_nockpt.txt 2>&1
```

4. **Facultatif:** nettoyer les artefacts de travail. Les exemples de commandes suivants montrent la suppression de l'objet de travail créé à l'étape 1.

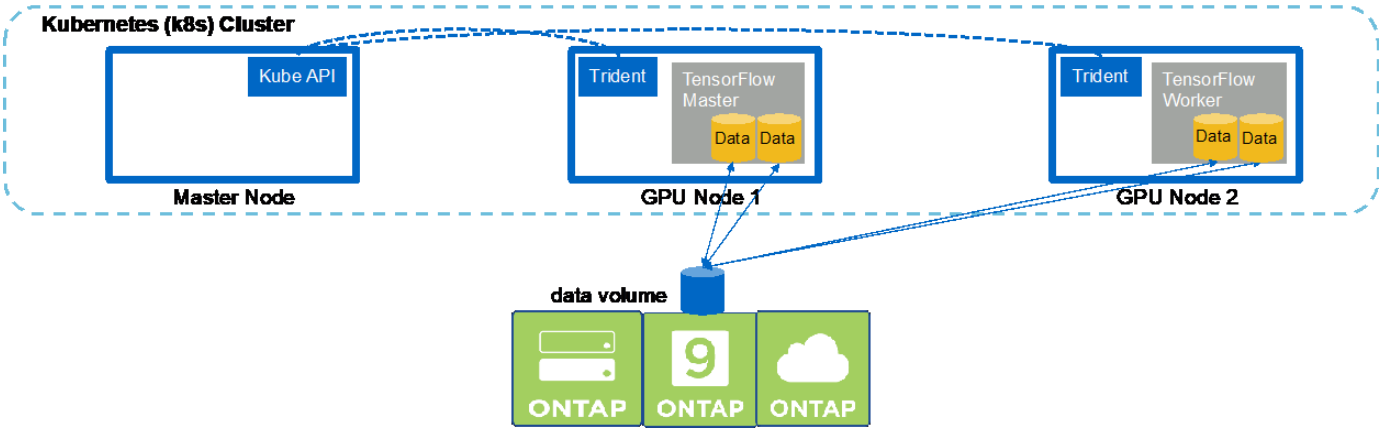
   Lorsque vous supprimez l'objet travail, Kubernetes supprime automatiquement les pods associés.

```
$ kubectl get jobs
NAME                                                COMPLETIONS   DURATION
AGE
netapp-tensorflow-single-imagenet                   1/1           5m42s
10m
$ kubectl get pods
NAME                                                     READY   STATUS
RESTARTS   AGE
netapp-tensorflow-single-imagenet-m7x92                  0/1     Completed
0          11m
$ kubectl delete job netapp-tensorflow-single-imagenet
job.batch "netapp-tensorflow-single-imagenet" deleted
$ kubectl get jobs
No resources found.
$ kubectl get pods
No resources found.
```

Exécutez un workload d'IA distribué synchrone

Pour exécuter une tâche d'IA et DE ML à plusieurs nœuds synchrones dans votre cluster Kubernetes, exécutez les tâches suivantes sur l'hôte de démarrage du déploiement. Ce processus vous permet de exploiter les données stockées sur un volume NetApp et d'utiliser plus de GPU que n'en fournir un seul nœud de travail. Reportez-vous à la figure suivante pour obtenir une description d'une tâche d'IA distribuée synchrone.

> Les tâches distribuées synchrones permettent d'améliorer les performances et la précision de l'entraînement par rapport aux tâches distribuées asynchrones. Un examen des avantages et inconvénients des emplois synchrones par rapport aux emplois asynchrones est hors du champ d'application de ce document.



1. Les commandes d'exemple suivantes montrent la création d'un travailleur qui participe à l'exécution distribuée synchrone du même travail de banc d'essai TensorFlow qui a été exécuté sur un seul nœud dans l'exemple de la section "Exécutez un workload d'IA à un seul nœud". Dans cet exemple spécifique, seul un travailleur est déployé car le travail est exécuté sur deux nœuds worker.

Cet exemple de déploiement utilisateur nécessite huit GPU, et peut donc s'exécuter sur un seul nœud worker GPU doté d'au moins huit GPU. Si les nœuds workers GPU disposent de plus de huit GPU, afin d'optimiser les performances, il est possible que vous souhaitiez augmenter ce nombre afin qu'il soit égal au nombre de GPU dont bénéficient les nœuds workers. Pour en savoir plus sur les déploiements Kubernetes, rendez-vous sur le "Documentation officielle Kubernetes".

Un déploiement Kubernetes est créé dans cet exemple, car ce travailleur conteneurisé ne s'en serait jamais achevé seul. C'est pourquoi il n'est pas logique de le déployer via la construction de tâches Kubernetes. Si votre travailleur est conçu ou écrit de manière à le compléter seul, il peut être judicieux d'utiliser la structure de travail pour déployer votre travailleur.

Le pod spécifié dans cet exemple de spécification de déploiement est donné un `hostNetwork` valeur de `true`. Cette valeur signifie que le pod utilise la pile réseau du nœud du worker hôte au lieu de la pile de réseau virtuel que Kubernetes crée habituellement pour chaque pod. Cette annotation est utilisée dans ce cas car la charge de travail spécifique repose sur Open MPI, NCCL et Horovod pour exécuter la charge de travail de façon synchrone distribuée. Par conséquent, elle nécessite l'accès à la pile réseau de l'hôte. Une discussion sur Open MPI, NCCL et Horovod n'est pas dans le cadre du présent document. Si cela est ou non `hostNetwork: true` l'annotation est nécessaire dépend des exigences de la charge de travail spécifique que vous exécutez. Pour plus d'informations sur le `hostNetwork` voir "Documentation officielle Kubernetes".

```
$ cat << EOF > ./netapp-tensorflow-multi-imagenet-worker.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: netapp-tensorflow-multi-imagenet-worker
spec:
  replicas: 1
  selector:
    matchLabels:
      app: netapp-tensorflow-multi-imagenet-worker
  template:
    metadata:
      labels:
        app: netapp-tensorflow-multi-imagenet-worker
    spec:
      hostNetwork: true
      volumes:
      - name: dshm
        emptyDir:
          medium: Memory
      - name: testdata-iface1
        persistentVolumeClaim:
          claimName: pb-fg-all-iface1
      - name: testdata-iface2
        persistentVolumeClaim:
          claimName: pb-fg-all-iface2
      - name: results
        persistentVolumeClaim:
```

```
          claimName: tensorflow-results
      containers:
      - name: netapp-tensorflow-py2
        image: netapp/tensorflow-py2:19.03.0
        command: ["bash", "/netapp/scripts/start-slave-multi.sh",
"22122"]
        resources:
          limits:
            nvidia.com/gpu: 8
        volumeMounts:
        - mountPath: /dev/shm
          name: dshm
        - mountPath: /mnt/mount_0
          name: testdata-iface1
        - mountPath: /mnt/mount_1
          name: testdata-iface2
        - mountPath: /tmp
          name: results
        securityContext:
          privileged: true
EOF
$ kubectl create -f ./netapp-tensorflow-multi-imagenet-worker.yaml
deployment.apps/netapp-tensorflow-multi-imagenet-worker created
$ kubectl get deployments
NAME                                      DESIRED   CURRENT   UP-TO-DATE
AVAILABLE   AGE
netapp-tensorflow-multi-imagenet-worker   1         1         1
1           4s
```

2. Confirmez que le déploiement de collaborateur que vous avez créé à l'étape 1 a été lancé avec succès. Les exemples de commandes suivants confirment qu'un seul pod worker a été créé pour le déploiement, comme indiqué dans la définition du déploiement, et que ce pod s'exécute actuellement sur l'un des nœuds workers GPU.

```
$ kubectl get pods -o wide
NAME                                                        READY
STATUS    RESTARTS   AGE
IP              NODE            NOMINATED NODE
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running   0          60s   10.61.218.154   10.61.218.154   <none>
$ kubectl logs netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725
22122
```

3. Créez un travail Kubernetes pour un master qui démarre, participe et suit l'exécution du travail multinœud synchrone. Les commandes d'exemple suivantes créent un master qui démarre, participe à et assure le suivi de l'exécution distribuée synchrone du même travail de banc d'essai TensorFlow qui a été exécuté

sur un seul nœud dans l'exemple de la section "Exécutez un workload d'IA à un seul nœud".

Cet exemple de tâche maître demande huit GPU, puis peut être exécuté sur un seul nœud worker GPU doté d'au moins huit GPU. Si les nœuds workers GPU disposent de plus de huit GPU, afin d'optimiser les performances, il est possible que vous souhaitiez augmenter ce nombre afin qu'il soit égal au nombre de GPU dont bénéficient les nœuds workers.

Le pod maître spécifié dans cet exemple de définition de travail est donné un `hostNetwork` valeur de `true`, tout comme le pod de travailleur a été donné `hostNetwork` valeur de `true` à l'étape 1. Voir l'étape 1 pour plus de détails sur la raison pour laquelle cette valeur est nécessaire.

```
$ cat << EOF > ./netapp-tensorflow-multi-imagenet-master.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: netapp-tensorflow-multi-imagenet-master
spec:
  backoffLimit: 5
  template:
    spec:
      hostNetwork: true
      volumes:
      - name: dshm
        emptyDir:
          medium: Memory
      - name: testdata-iface1
        persistentVolumeClaim:
          claimName: pb-fg-all-iface1
      - name: testdata-iface2
        persistentVolumeClaim:
          claimName: pb-fg-all-iface2
      - name: results
        persistentVolumeClaim:
          claimName: tensorflow-results
      containers:
      - name: netapp-tensorflow-py2
        image: netapp/tensorflow-py2:19.03.0
        command: ["python", "/netapp/scripts/run.py", "--
dataset_dir=/mnt/mount_0/dataset/imagenet", "--port=22122", "--
num_devices=16", "--dgx_version=dgx1", "--
nodes=10.61.218.152,10.61.218.154"]
        resources:
          limits:
            nvidia.com/gpu: 8
        volumeMounts:
        - mountPath: /dev/shm
          name: dshm
```

```
        - mountPath: /mnt/mount_0
          name: testdata-iface1
        - mountPath: /mnt/mount_1
          name: testdata-iface2
        - mountPath: /tmp
          name: results
        securityContext:
          privileged: true
      restartPolicy: Never
EOF
$ kubectl create -f ./netapp-tensorflow-multi-imagenet-master.yaml
job.batch/netapp-tensorflow-multi-imagenet-master created
$ kubectl get jobs
NAME                                      COMPLETIONS   DURATION   AGE
netapp-tensorflow-multi-imagenet-master   0/1           25s        25s
```

4. Vérifiez que le travail principal que vous avez créé à l'étape 3 fonctionne correctement. L'exemple de commande suivant confirme qu'un module maître unique a été créé pour le travail, comme indiqué dans la définition du travail, et que ce pod s'exécute actuellement sur l'un des nœuds workers GPU. Vous devriez également voir que le pod de worker que vous avez initialement vu à l'étape 1 est toujours en cours d'exécution et que les pods master et worker exécutent sur différents nœuds.

```
$ kubectl get pods -o wide
NAME                                                        READY
STATUS    RESTARTS   AGE
IP              NODE            NOMINATED NODE
netapp-tensorflow-multi-imagenet-master-ppwwj               1/1
Running   0          45s   10.61.218.152   10.61.218.152   <none>
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running   0          26m   10.61.218.154   10.61.218.154   <none>
```

5. Confirmez que le travail principal que vous avez créé à l'étape 3 s'est terminé avec succès. L'exemple de commandes suivant confirme que le travail a été terminé avec succès.

```
$ kubectl get jobs
NAME                                      COMPLETIONS   DURATION   AGE
netapp-tensorflow-multi-imagenet-master   1/1           5m50s      9m18s
$ kubectl get pods
NAME                                                        READY
STATUS      RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj               0/1
Completed   0          9m38s
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running     0          35m
$ kubectl logs netapp-tensorflow-multi-imagenet-master-ppwwj
```

```
[10.61.218.152:00008] WARNING: local probe returned unhandled
shell:unknown assuming bash
rm: cannot remove '/lib': Is a directory
[10.61.218.154:00033] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 702
[10.61.218.154:00033] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 711
[10.61.218.152:00008] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 702
[10.61.218.152:00008] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 711
Total images/sec = 12881.33875
================ Clean Cache !!! ==================
mpirun -allow-run-as-root -np 2 -H 10.61.218.152:1,10.61.218.154:1 -mca
pml ob1 -mca btl ^openib -mca btl_tcp_if_include enp1s0f0 -mca
plm_rsh_agent ssh -mca plm_rsh_args "-p 22122" bash -c 'sync; echo 1 >
/proc/sys/vm/drop_caches'
==========================================
mpirun -allow-run-as-root -np 16 -H 10.61.218.152:8,10.61.218.154:8
-bind-to none -map-by slot -x NCCL_DEBUG=INFO -x LD_LIBRARY_PATH -x PATH
-mca pml ob1 -mca btl ^openib -mca btl_tcp_if_include enp1s0f0 -x
NCCL_IB_HCA=mlx5 -x NCCL_NET_GDR_READ=1 -x NCCL_IB_SL=3 -x
NCCL_IB_GID_INDEX=3 -x
NCCL_SOCKET_IFNAME=enp5s0.3091,enp12s0.3092,enp132s0.3093,enp139s0.3094
-x NCCL_IB_CUDA_SUPPORT=1 -mca orte_base_help_aggregate 0 -mca
plm_rsh_agent ssh -mca plm_rsh_args "-p 22122" python
/netapp/tensorflow/benchmarks_190205/scripts/tf_cnn_benchmarks/tf_cnn_be
nchmarks.py --model=resnet50 --batch_size=256 --device=gpu
--force_gpu_compatible=True --num_intra_threads=1 --num_inter_threads=48
--variable_update=horovod --batch_group_size=20 --num_batches=500
--nodistortions --num_gpus=1 --data_format=NCHW --use_fp16=True
--use_tf_layers=False --data_name=imagenet --use_datasets=True
--data_dir=/mnt/mount_0/dataset/imagenet
--datasets_parallel_interleave_cycle_length=10
--datasets_sloppy_parallel_interleave=False --num_mounts=2
--mount_prefix=/mnt/mount_%d --datasets_prefetch_buffer_size=2000 --
datasets_use_prefetch=True --datasets_num_private_threads=4
--horovod_device=gpu >
/tmp/20190814_161609_tensorflow_horovod_rdma_resnet50_gpu_16_256_b500_im
agenet_nodistort_fp16_r10_m2_nockpt.txt 2>&1
```

6. Supprimez le déploiement de collaborateur lorsque vous n'en avez plus besoin. L'exemple de commandes suivant montre la suppression de l'objet de déploiement de travail qui a été créé à l'étape 1.

   Lorsque vous supprimez l'objet de déploiement worker, Kubernetes supprime automatiquement les pods workers associés.

```
$ kubectl get deployments
NAME                                        DESIRED   CURRENT   UP-TO-DATE
AVAILABLE   AGE
netapp-tensorflow-multi-imagenet-worker   1         1         1
1           43m
$ kubectl get pods
NAME                                                        READY
STATUS      RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj              0/1
Completed   0          17m
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running     0          43m
$ kubectl delete deployment netapp-tensorflow-multi-imagenet-worker
deployment.extensions "netapp-tensorflow-multi-imagenet-worker" deleted
$ kubectl get deployments
No resources found.
$ kubectl get pods
NAME                                            READY   STATUS
RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj   0/1     Completed   0
18m
```

7. **Facultatif:** nettoyez les artefacts du travail principal. Les exemples de commandes suivants montrent la suppression de l'objet de travail maître créé à l'étape 3.

   Lorsque vous supprimez l'objet de travail maître, Kubernetes supprime automatiquement les modules maîtres associés.

```
$ kubectl get jobs
NAME                                        COMPLETIONS   DURATION   AGE
netapp-tensorflow-multi-imagenet-master   1/1           5m50s      19m
$ kubectl get pods
NAME                                            READY   STATUS
RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj   0/1     Completed   0
19m
$ kubectl delete job netapp-tensorflow-multi-imagenet-master
job.batch "netapp-tensorflow-multi-imagenet-master" deleted
$ kubectl get jobs
No resources found.
$ kubectl get pods
No resources found.
```

Test des performances

Nous avons effectué une comparaison simple des performances dans le cadre de la création de cette solution. Nous avons exécuté plusieurs tâches standard de banc d'essai NetApp ai avec Kubernetes, et nous avons comparé les résultats du banc d'essai avec les sous-activités qui ont été réalisées à l'aide d'une simple commande Docker run. Nous n'avons pas constaté de différence notable au niveau des performances. Nous en avons donc conclu que l'utilisation de Kubernetes pour orchestrer les tâches d'entraînement d'IA conteneurisées n'avait pas de conséquences négatives sur les performances. Pour consulter les résultats de cette comparaison, consultez le tableau suivant.

Banc d'essai	Jeu de données	Docker Run (images/s)	Kubernetes (images/s)
TensorFlow à un nœud	Données synthétiques	6,667.2475	6,661.93125
TensorFlow à un nœud	ImageNet	6,570.2025	6,530.59125
TensorFlow distribué synchrone à deux nœuds	Données synthétiques	13,213.70625	13,218.288125
TensorFlow distribué synchrone à deux nœuds	ImageNet	12,941.69125	12,881.33875

Conclusion

Toutes les entreprises, quelle que soit leur taille et tous les secteurs, se tournent vers l'intelligence artificielle (IA), le machine learning (ML) et le deep learning (DL) pour résoudre des problèmes concrets, proposer des produits et des services innovants et se démarquer sur un marché de plus en plus concurrentiel. Alors que les entreprises ont de plus en plus recours à l'IA, AU ML et au DL, elles sont confrontées à de nombreux défis, notamment l'évolutivité des workloads et la disponibilité des données. Ces difficultés peuvent être résolues via l'utilisation de la solution NetApp ai Control plane.

Cette solution permet de cloner rapidement un espace de noms de données. Vous pouvez par ailleurs définir et implémenter des workflows d'entraînement d'IA, DE ML et de DL pour intégrer la création quasi instantanée de données et de modèles de base à des fins de traçabilité et de gestion des versions. Avec cette solution, vous pouvez suivre l'entraînement de chaque modèle individuel sur le(s) dataset(s) exact(s) auquel le modèle a été entraîné et/ou validé. Enfin, cette solution vous permet de provisionner rapidement des espaces de travail Jupyter Notebook avec un accès à des jeux de données volumineux.

Cette solution cible les data Scientists et les ingénieurs de données, une expertise minimale de NetApp ou de NetApp ONTAP est requise. Avec cette solution, les fonctions de gestion des données peuvent être exécutées à l'aide d'outils et d'interfaces simples et familiers. De plus, cette solution utilise des composants entièrement open source et libres. Par conséquent, si vous disposez déjà d'un système de stockage NetApp dans votre environnement, vous pouvez implémenter cette solution dès aujourd'hui. Si vous souhaitez essayer cette solution, mais que vous ne disposez pas encore de système de stockage NetApp, rendez-vous sur "cloud.netapp.com", Et vous pouvez être opérationnel avec une solution de stockage NetApp basée sur le cloud en un rien de temps.

Informations sur le copyright

Copyright © 2024 NetApp, Inc. Tous droits réservés. Imprimé aux États-Unis. Aucune partie de ce document protégé par copyright ne peut être reproduite sous quelque forme que ce soit ou selon quelque méthode que ce soit (graphique, électronique ou mécanique, notamment par photocopie, enregistrement ou stockage dans un système de récupération électronique) sans l'autorisation écrite préalable du détenteur du droit de copyright.

Les logiciels dérivés des éléments NetApp protégés par copyright sont soumis à la licence et à l'avis de non-responsabilité suivants :

CE LOGICIEL EST FOURNI PAR NETAPP « EN L'ÉTAT » ET SANS GARANTIES EXPRESSES OU TACITES, Y COMPRIS LES GARANTIES TACITES DE QUALITÉ MARCHANDE ET D'ADÉQUATION À UN USAGE PARTICULIER, QUI SONT EXCLUES PAR LES PRÉSENTES. EN AUCUN CAS NETAPP NE SERA TENU POUR RESPONSABLE DE DOMMAGES DIRECTS, INDIRECTS, ACCESSOIRES, PARTICULIERS OU EXEMPLAIRES (Y COMPRIS L'ACHAT DE BIENS ET DE SERVICES DE SUBSTITUTION, LA PERTE DE JOUISSANCE, DE DONNÉES OU DE PROFITS, OU L'INTERRUPTION D'ACTIVITÉ), QUELLES QU'EN SOIENT LA CAUSE ET LA DOCTRINE DE RESPONSABILITÉ, QU'IL S'AGISSE DE RESPONSABILITÉ CONTRACTUELLE, STRICTE OU DÉLICTUELLE (Y COMPRIS LA NÉGLIGENCE OU AUTRE) DÉCOULANT DE L'UTILISATION DE CE LOGICIEL, MÊME SI LA SOCIÉTÉ A ÉTÉ INFORMÉE DE LA POSSIBILITÉ DE TELS DOMMAGES.

NetApp se réserve le droit de modifier les produits décrits dans le présent document à tout moment et sans préavis. NetApp décline toute responsabilité découlant de l'utilisation des produits décrits dans le présent document, sauf accord explicite écrit de NetApp. L'utilisation ou l'achat de ce produit ne concède pas de licence dans le cadre de droits de brevet, de droits de marque commerciale ou de tout autre droit de propriété intellectuelle de NetApp.

Le produit décrit dans ce manuel peut être protégé par un ou plusieurs brevets américains, étrangers ou par une demande en attente.

LÉGENDE DE RESTRICTION DES DROITS : L'utilisation, la duplication ou la divulgation par le gouvernement sont sujettes aux restrictions énoncées dans le sous-paragraphe (b)(3) de la clause Rights in Technical Data-Noncommercial Items du DFARS 252.227-7013 (février 2014) et du FAR 52.227-19 (décembre 2007).

Les données contenues dans les présentes se rapportent à un produit et/ou service commercial (tel que défini par la clause FAR 2.101). Il s'agit de données propriétaires de NetApp, Inc. Toutes les données techniques et tous les logiciels fournis par NetApp en vertu du présent Accord sont à caractère commercial et ont été exclusivement développés à l'aide de fonds privés. Le gouvernement des États-Unis dispose d'une licence limitée irrévocable, non exclusive, non cessible, non transférable et mondiale. Cette licence lui permet d'utiliser uniquement les données relatives au contrat du gouvernement des États-Unis d'après lequel les données lui ont été fournies ou celles qui sont nécessaires à son exécution. Sauf dispositions contraires énoncées dans les présentes, l'utilisation, la divulgation, la reproduction, la modification, l'exécution, l'affichage des données sont interdits sans avoir obtenu le consentement écrit préalable de NetApp, Inc. Les droits de licences du Département de la Défense du gouvernement des États-Unis se limitent aux droits identifiés par la clause 252.227-7015(b) du DFARS (février 2014).

Informations sur les marques commerciales

NETAPP, le logo NETAPP et les marques citées sur le site http://www.netapp.com/TM sont des marques déposées ou des marques commerciales de NetApp, Inc. Les autres noms de marques et de produits sont des marques commerciales de leurs propriétaires respectifs.